Final Report: Data Driven Support for Cyber-Physical Systems

Kernel-Offloaded DDoS Detection with eBPF and XDP

Towards Multi-Layer Intrusion Detection Systems

IDA ÅBERG
ANDRZEJ PULTYN
MÅRTEN GRANATH
TABITA TENGBLAD

Chalmers University of Technology
Gothenburg, Sweden 2026

Abstract

Distributed denial-of-service (DDoS) attacks keep growing in scale. This necessitates automated defensive mechanisms to detect and drop malicious traffic on hosts that are targeted. A userspace detector must receive every packet through the kernel's network stack before counting it, so it competes for the resources an attack aims to exhaust. This work investigates whether moving per-packet counting and heavy-hitter tracking into the Linux kernel, using an eBPF program at the XDP hook, reduces the cost of detection without degrading its quality. Building on earlier work, a HeavyKeeper sketch tracks the heaviest sources in each time window. An adaptive threshold based on the interquartile range is then evaluated in userspace. The kernel-offloaded implementation was compared with an equivalent userspace baseline on a Raspberry Pi 400 and a FABRIC virtual machine, using replayed CIC-DDoS2019 traffic and XDP in generic mode.

On FABRIC, both implementations counted nearly all packets. There, host CPU time per counted packet fell from 6.69 to 2.73 µs, with nearly identical detection quality (F1 0.954 versus 0.951). On the Raspberry Pi, the baseline counted 26.75% of received packets against 93.36% for the kernel implementation. CPU time per counted packet fell roughly tenfold, and recall rose from 76% to 89%. Memory use and decision delay were essentially unchanged, and with blocking enabled both performed similarly. The results are based on one run per configuration and a single attack type. They nonetheless indicate that in-kernel heavy-hitter tracking is feasible and substantially lowers the cost of volumetric DDoS detection, especially on constrained devices. It is best seen as a first layer shielding a multi-layer intrusion detection system from the bulk of a flood.

Keywords: DDoS detection, network intrusion detection, eBPF, XDP, heavy-hitter detection, data sketches, HeavyKeeper, interquartile range, kernel-level packet processing.

Acknowledgments

We would like to thank our supervisor, Romaric Duvignau, for his guidance, support, and valuable feedback throughout this project. His insights and encouragement have been greatly appreciated and have contributed to both our progress and understanding of the subject.

We would also like to thank Christian Van Sloun for his continued involvement, helpful discussions, and valuable perspectives throughout the project.

Glossary

This section lists the acronyms and technical terms used throughout this thesis, in alphabetical order.

Acronyms

AI	Artificial Intelligence
BPF	Berkeley Packet Filter
CAIDA	Center for Applied Internet Data Analysis
CIC	Canadian Institute for Cybersecurity
CPU	Central Processing Unit
CSV	Comma-Separated Values
DDoS	Distributed Denial of Service
DNS	Domain Name System
eBPF	extended Berkeley Packet Filter
F1	F1 score
FIFO	First In, First Out
Gbps/Tbps	gigabits/terabits per second
IDS	Intrusion Detection System
IoT	Internet of Things
IP	Internet Protocol
IPv4/IPv6	Internet Protocol version 4/6
IQR	Interquartile Range
LDAP	Lightweight Directory Access Protocol
LRU	Least Recently Used
ML	Machine Learning
NAT	Network Address Translation
NIC	Network Interface Card
NIDS	Network Intrusion Detection System
NTP	Network Time Protocol
OS	Operating System
PCAP	Packet Capture
SYN	Synchronize (TCP connection request)
TAI	International Atomic Time
TCP	Transmission Control Protocol
UDP	User Datagram Protocol
vCPU	virtual Central Processing Unit
VM	Virtual Machine
XDP	eXpress Data Path

Terms

`AF_PACKET`	Linux raw socket type that delivers a copy of each packet at the link layer to user space. Used by the baseline to capture packets.
`AF_XDP`	Linux socket type that lets an XDP program redirect packets directly into user-space memory, bypassing most of the network stack.

cgroup	Control group. A Linux kernel mechanism that groups processes and accounts for, or limits, their resource usage, such as memory.
Elephant flow	A flow that carries a large share of the traffic. See *heavy hitter*.
Epoch	The index of a fixed-length time window, computed as $\lfloor t/L \rfloor$ for a timestamp t and window length L.
False positive rate	The share of benign windows that are incorrectly flagged as malicious.
Fingerprint	A short hash of a source address, stored in each HeavyKeeper bucket to identify which flow holds it.
Generic mode	XDP mode in which the program runs after the socket buffer has been allocated. Works with any network driver, but gives up most of the performance advantage of XDP.
Goroutine	A lightweight thread managed by the Go runtime.
Heavy hitter	A source that accounts for an unusually large share of the traffic within a time window.
Mouse flow	A flow that carries only a small amount of traffic. HeavyKeeper deliberately lets these pass without tracking them.
Native mode	XDP mode in which the program runs in the network driver, before a socket buffer is allocated. Requires driver support.
Pane	A short, non-overlapping sub-window whose partial results are combined to form sliding windows.
Precision	The share of windows flagged as malicious that actually are malicious.
Recall	The share of malicious windows that are flagged as malicious.
Slice	In FABRIC, a set of reserved resources, such as virtual machines and network links, that together form one experiment.
Sliding window	A window that overlaps its predecessor, with a new window starting at every slide interval.
Socket buffer (`sk_buff`)	The per-packet data structure that the Linux network stack uses to represent a packet.
Software interrupt (softirq)	The deferred kernel context in which network receive processing, including XDP, runs. Its CPU time is not charged to any user-space process.
Top-K	The K heaviest sources in a window, as tracked by HeavyKeeper ($K = 10$ in this work).

Tumbling window	Consecutive, non-overlapping windows of fixed length, so that every packet belongs to exactly one window.
Verifier	The Linux kernel component that checks an eBPF program before it is loaded, ensuring that it terminates and only accesses memory safely.
Virtual function	A lightweight virtual instance of a physical network card, assigned directly to a virtual machine.
Warm-up	The initial phase in which a detector history holds fewer than the minimum number of samples (four by default). All verdicts during this phase are benign.

Contents

1 Introduction **1**
1.1 Scope and Delimitations . . . 1
1.1.1 Primary Objective . . . 1
1.1.2 In-Scope . . . 1
1.1.3 Out-of-Scope . . . 2
1.2 Related Work . . . 2
1.2.1 Wickman and Rygaard (2025) . . . 2
1.2.2 Koshovyi and Wu (2026) . . . 2
1.2.3 Summary . . . 3

2 Background **4**
2.1 eBPF . . . 4
2.2 XDP . . . 4
2.3 Data Sketches . . . 4
2.3.1 Count-Min Sketch . . . 5
2.3.2 HeavyKeeper Algorithm . . . 5
2.4 Interquartile Range Rule . . . 6
2.5 Windowing . . . 6

3 Methodology **8**
3.1 Motivating Model Architecture . . . 8
3.2 Experiment Design . . . 8
3.2.1 Hardware Testbeds . . . 8
3.2.2 Dataset . . . 9
3.2.3 Traffic Replay . . . 9
3.3 Experiment Procedure . . . 9
3.3.1 Selected traffic excerpt . . . 9
3.3.2 Test Configurations and Execution . . . 10
3.3.3 Data Collection . . . 10
3.3.4 Detection Evaluation and Temporal Alignment . . . 11

4 Implementation **12**
4.1 Baseline Implementation . . . 12
4.1.1 Flow . . . 13
4.1.2 HeavyKeeper . . . 13
4.1.3 IQR . . . 15
4.2 Kernel-Offloaded Implementation . . . 17
4.2.1 BPF Maps . . . 17
4.2.2 HeavyKeeper . . . 18
4.3 Window mechanism . . . 19
4.3.1 Common design . . . 19
4.3.2 User space . . . 19
4.3.3 Kernel space . . . 20

5 Results **21**
5.1 Blocking Disabled . . . 21
5.1.1 Packet Accounting and Loss . . . 21
5.1.2 CPU Utilization . . . 24
5.1.3 Detection Quality . . . 25
5.2 Blocking Enabled . . . 28

5.2.1 Memory and Decision Delay 28
5.3 Summary 30

6 Discussion **31**
6.1 Key Findings 31
6.2 Evaluation 32
6.3 Limitations 34
6.4 Future Work 34

7 Conclusion **36**

A Source Code **39**

B Dataset **39**

C TCPreplay configurations **39**

D Final Presentation Slides **39**

1 Introduction

DDoS attacks remain cheap and easy to launch, and their scale keeps growing: Cloudflare's DDoS threat report for the first half of 2026 records 805 attacks exceeding 1 Tbps in the second quarter alone, more than six times as many as in the first [1]. Most attacks are also short, with over 90% ending within ten minutes, which leaves no time for manual intervention [1]. Detection must therefore be automated and run continuously on the hosts it protects. A detector in user space, however, needs every packet to pass through the kernel's network stack and be copied to user space before it can even be counted, so that the detector itself competes for the resources an attack aims to exhaust. In the NIDS this work builds on [7], XDP is used only to enforce a blocklist, while every other packet is still forwarded to user space for analysis. This work instead moves the per-packet counting and heavy-hitter tracking into the XDP program, leaving only the per-window decision in user space, and investigates whether this reduces the resources spent on detection without degrading its quality.

1.1 Scope and Delimitations

This section defines the technical boundaries of the project: its primary objective, what the work covers, and what it deliberately leaves out.

1.1.1 Primary Objective

The primary objective of this project is to investigate whether moving the per-packet counting and heavy-hitter tracking of a volumetric DDoS detector from user space into the Linux kernel, using an eBPF program attached to the XDP hook, reduces the resources spent on detection without degrading its quality. This is broken down into two questions:

1. How do CPU utilization, memory usage, packet loss and decision delay differ between the kernel-offloaded implementation and an equivalent user-space baseline?
2. Do the two implementations detect attacks equally well, and is their detection quality comparable to that reported for the statistical baseline by Koshovyi and Wu [7]?

1.1.2 In-Scope

The project covers the following:

- **A user-space baseline.** A re-implementation in Go of the statistical detector of Wickman and Rygaard [6], combining HeavyKeeper with an adaptive threshold based on the interquartile range, extended to process live traffic captured from a network interface. In the baseline, the XDP program only enforces a blocklist of source addresses.
- **A kernel-offloaded implementation.** An eBPF/XDP program that performs the per-packet work in the kernel, maintaining the window totals, the HeavyKeeper sketch and the top-K candidates in BPF maps, with the algorithms adapted to the restrictions of the eBPF verifier. Userspace reads only a summary of each window from the maps, computes the verdicts and updates the blocklist enforced by the XDP program.
- **Equivalence between the two.** Both implementations share the same detection model, address representation and central configuration parameters. Differences in packet processing and data structures are documented in Chapter 4 and considered when interpreting the results.
- **Evaluation.** Replaying excerpts of the CIC-DDoS2019 dataset with `tcpreplay` onto a real network interface, on a Raspberry Pi 400 and a FABRIC virtual machine with XDP in generic mode. Both implementations are measured with blocking disabled for their detection quality

tests, while it is tested with blocking both turned off and turned on for system performance tests (Section 3.3.3).

1.1.3 Out-of-Scope

Fully In-Kernel Detection Only the per-packet counting is moved into the kernel. The IQR threshold and the resulting verdicts are still computed in userspace once per window, which means that blocking takes effect from the window after an attack is detected.

Integration with the Existing NIDS The detector is evaluated as a standalone system. Connecting it to the machine learning models of the earlier NIDS [7], for example by forwarding suspicious traffic to them, is not part of this work, and neither is developing, training or fine-tuning machine learning models.

Non-Volumetric and Signature-Based Detection The project targets volumetric attacks in which individual source addresses stand out as heavy hitters. Low-rate and application-layer attacks, as well as signature-based detection of known attacks, are out of scope.

Mitigation Beyond Source Blocking Mitigation is limited to dropping packets from blocklisted source addresses. Rate limiting, upstream filtering and other mitigation strategies are not considered.

Optimized User-Space Packet Capture The baseline receives packets through an `AF_PACKET` socket, one packet per system call. Faster capture mechanisms, such as memory-mapped rings or `AF_XDP`, are not explored, so the baseline is a straightforward reference rather than the fastest possible userspace implementation.

Maximum Load Both implementations are compared under the load produced by replaying the dataset with its original timing. The highest packet rate each implementation can sustain is not measured.

1.2 Related Work

This work builds on a NIDS developed in two previous Chalmers master's theses in collaboration with Ericsson [6, 7], described below.

1.2.1 Wickman and Rygaard (2025)

Wickman and Rygaard [6] developed an anomaly-based NIDS for volumetric DDoS detection with low memory and computational cost. Their system aggregates traffic in one-second sliding windows, estimates per-source packet counts with a Count-Min Sketch, tracks the heaviest sources with HeavyKeeper, and flags a window as malicious when its packet count exceeds an adaptive threshold derived from the interquartile range of previous windows. Evaluated on Ericsson's normal traffic with attack traces injected on top, it reached a detection accuracy of 96.09% on attacks from the CAIDA DDoS 2007 dataset, with no false positives on benign-only traffic. All processing took place in user space, with traffic replayed inside the application rather than received from a network interface. Our userspace baseline re-implements their statistical configuration, leaving out the Count-Min Sketch for the reasons given in Section 3.1.

1.2.2 Koshovyi and Wu (2026)

Koshovyi and Wu [7] extended this NIDS with an Isolation Forest trained on flow features, and compared monolithic and microservice-based deployments. Their system was evaluated on a Raspberry

Pi 5, with CIC-DDoS2019 replayed over the network and window-level ground truth derived from the dataset's flow labels: a window is labelled malicious if it overlaps any malicious flow. On an excerpt of 1,533 windows, the statistical baseline achieved a precision close to 1.0 but a recall of about 0.40, whereas the Isolation Forest reached an accuracy of 0.93. To process real traffic, they attached an XDP program that drops packets from blocklisted source addresses and forwards the metadata of every other packet to user space, where all detection takes place. To allow a loose comparison with their results, we use the same ground truth on CIC-DDoS2019.

1.2.3 Summary

Both previous theses perform all detection in user space: Wickman and Rygaard process replayed traffic within the application, and Koshovyi and Wu use XDP only to enforce decisions made in user space. This work instead moves the per-packet counting and heavy-hitter tracking into the XDP program, leaving only the per-window threshold evaluation in user space, and compares it with an equivalent user-space implementation. A broader survey of eBPF/XDP-based DDoS defense was outside the scope of this project.

2 Background

This chapter describes important theoretical concepts that our methodology and implementation build on. It starts with introducing technologies used in the project, the eBPF and XDP (Sections 2.1 & 2.2). Then it explains the algorithms and methods used to solve the problem, the data sketches (Section 2.3), the interquartile range rule (Section 2.4) and windowing mechanism (Section 2.5).

2.1 eBPF

eBPF, or **extended Berkeley Packet Filter**, is a technology for running sandboxed programs inside the Linux kernel without modifying its source code or loading kernel modules. A program is written in a restricted subset of C, compiled to eBPF bytecode, and attached to a hook in the kernel, such as the XDP hook described in Section 2.2, where it runs every time the corresponding event occurs. Before the program is loaded, it is checked by the verifier, and it is then compiled to native machine code, so that it runs at close to the speed of compiled kernel code [16].

The verifier ensures that a program cannot crash or stall the kernel, which places several restrictions on what it may do. Every memory access, including every read of packet data, must be proven to lie within bounds, and the program must be proven to terminate: loops are only accepted if their number of iterations is bounded [16], and the verifier analyses at most one million instructions per program. The stack is limited to 512 bytes [17], the program can only call a fixed set of helper functions provided by the kernel, for example to obtain random numbers or the current time, and there are no floating-point instructions.

A program keeps no state between invocations except in maps, which are data structures held in the kernel and accessible both from eBPF programs and from user space. Maps come in several types, such as arrays and hash tables. Since the same program runs in parallel on every CPU that receives packets, updates to a shared map must be atomic. Alternatively, per-CPU variants of the map types give each CPU its own copy, which avoids contention, but requires user space to combine the values of all CPUs when reading them [16].

2.2 XDP

XDP, or **eXpress Data Path** [9], is a hook that runs an eBPF program on every packet a network interface receives, at the earliest point in the kernel's receive path. The program inspects the raw packet and returns an action, most commonly to drop it (`XDP_DROP`) or to pass it on to the regular network stack (`XDP_PASS`). Packets that are passed on are processed as usual, so XDP does not replace the network stack but lets a program act on packets before the stack does. Like other eBPF programs, it can share state with user space through BPF maps (Section 2.1).

In native mode, the program is run by the network driver before the kernel allocates a socket buffer (`sk_buff`), the per-packet structure the rest of the network stack relies on. A packet that reaches a user-space program must in addition be copied to a socket. Handling a packet in XDP therefore avoids most of the per-packet cost of the stack [9]. If the driver lacks support for native mode, the program can run in generic mode instead, after the socket buffer has been allocated, which works with any interface but gives up most of this advantage. In both modes, the program runs in the kernel's software-interrupt context, so its processing time is not accounted to any user-space process.

2.3 Data Sketches

A challenge in analyzing data streams with heavy traffic is handling the large volume of data in an efficient way. One strategy is to use data sketches, which provide fast and space efficient tracking at the cost of bounded approximation errors. For this project, two metrics need to be tracked in order to detect a DDoS attack: which IP addresses generate the most traffic, and how many packets those

addresses have sent. This section first describes the Count-Min Sketch, a classic sketch for estimating how often each item occurs in a stream, and then HeavyKeeper, which is the sketch used in this project.

2.3.1 Count-Min Sketch

The Count-Min Sketch [5] estimates how many times an item has been observed in a stream, which in network traffic is how many packets each source address has sent [6]. It consists of d rows of w counters, each row with its own hash function. Every packet increments one counter per row, and the estimate for an address is the smallest of its d counters (Figure 1).

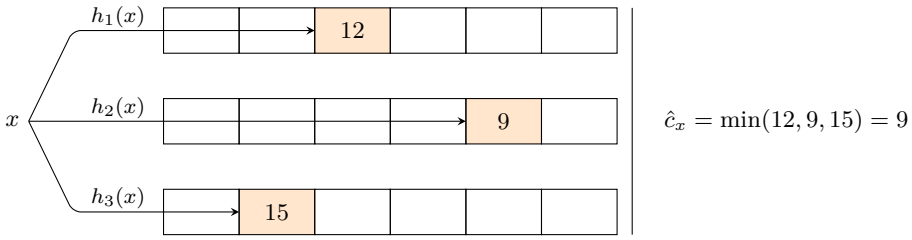


Figure 1: A Count-Min Sketch with $d = 3$ rows of $w = 6$ counters. An address x is hashed to one counter per row, each of which is incremented when a packet from x arrives. Since the counters are shared with other addresses, the estimate of x is the smallest of them.

Since far more addresses than counters exist, every counter is shared by several addresses and may hold their combined traffic. Using several independent rows means that an address colliding with heavy traffic in one row is unlikely to do so in all of them, so the smallest counter is the one least affected by other flows. The sketch can therefore overestimate a count, but never underestimate it [5]. Updates and queries touch one counter per row, and since d and w are fixed, both take constant time and the memory is constant at $O(d \cdot w)$, which makes the structure suitable for real-time traffic monitoring [6].

For heavy-hitter detection, the Count-Min Sketch has two drawbacks. It stores counts but no identities, so it can estimate the size of an address presented to it but cannot list which addresses are the largest. And every address contributes to the counters, so the many small flows of normal traffic inflate the estimates of the large ones. HeavyKeeper, described next, addresses both [4].

2.3.2 HeavyKeeper Algorithm

The HeavyKeeper algorithm identifies the top-K flows in a data stream and estimates their sizes. Unlike the Count-Min Sketch, it does not attempt to track every flow: small flows, commonly referred to as mouse flows, are deliberately discarded so that the available memory can be spent on the flows that actually matter [4].

Like the Count-Min Sketch, HeavyKeeper is built around a $d \cdot w$ matrix with a hash function assigned to each row. The difference lies in the contents of the cells: each cell holds a counter and a fingerprint. The fingerprint is produced by a separate hash function and truncated to a fixed-size identifier for the IP address. It is the same in every row, since it represents the identity of the flow rather than its location.

When a packet arrives, the address is hashed once per row to obtain one cell in each row. Before the counter of a cell can be modified, the fingerprint stored in the cell is compared to the fingerprint of the packet, which gives three possible cases:

1. **The cell is empty**: no flow has been assigned to the cell yet. The incoming address writes its own fingerprint into the cell and sets the counter to 1.

2. **The fingerprints are the same**: the cell is currently held by the incoming flow, and the counter is incremented by 1.
3. **The fingerprints differ**: the cell is held by another flow. This triggers a probabilistic decrease of the counter, where a weighted dice roll decides whether the counter is decremented by 1 or left untouched. The probability of a decrease is b^{-C}, where b is a predefined coefficient slightly larger than 1 and C is the current value of the counter. If the counter reaches 0 as a result, the previous flow has been evicted, and the incoming address takes over the cell by writing its own fingerprint and setting the counter to 1.

Since the decay probability falls exponentially with the counter, a flow holding a cell with a low counter is decayed by almost every competing packet and is evicted after only a few collisions, whereas a flow with a high counter is left virtually untouched. Mouse flows therefore pass through the structure without settling, while elephant flows hold on to their cells [4].

Querying the size of a flow follows the same addressing scheme. The d cells of the address are read, those whose fingerprint does not match are discarded as belonging to other flows, and the largest of the remaining counters is reported. If no cell matches, the flow is held nowhere and is reported as a mouse flow. The maximum is used because decay can only push a counter below the true size of a flow and never above it, so the reported value comes from the row in which the flow has been least disturbed.

2.4 Interquartile Range Rule

The interquartile range (IQR) rule is a statistical method for identifying outliers in a dataset. It is based on the spread of the middle half of the observations, which makes it robust against extreme values and does not require the data to follow a normal distribution.

The interquartile range is defined as the difference between the third and the first quartile:

$$\mathrm{IQR} = Q_3 - Q_1 \tag{1}$$

An observation is considered an outlier if it falls outside the range spanned by the quartiles extended by a multiple of the IQR. Using a factor of 1.5, the bounds are

$$x < Q_1 - 1.5 \cdot \mathrm{IQR} \qquad \text{or} \qquad x > Q_3 + 1.5 \cdot \mathrm{IQR} \tag{2}$$

while a factor of 3 instead identifies more extreme outliers.

Because the quartiles are unaffected by how far away the most extreme observations lie, a few very large values cannot inflate the bounds. This makes the rule suitable for network traffic, where the volume fluctuates naturally and an attack produces values far outside the ordinary range. The rule alone does not, however, prevent a sustained attack from shifting the quartiles; how the detector keeps attack traffic out of its history is described in Section 4.1.3.

In this project only the upper bound in Equation 2 is of interest, since an unusually low packet count does not indicate an attack.

2.5 Windowing

Since a data stream has no end, a window mechanism is needed to aggregate the incoming data into bounded intervals that statistics can be computed over [14]. Dividing the stream this way also bounds the memory required, as the data collected during an interval can be discarded once that interval has been processed.

The simplest form is the **tumbling window**, where the stream is divided into consecutive intervals of fixed length and every packet belongs to exactly one interval (Figure 2). For DDoS detection this has

a drawback: an attack lasting from 1.8 s to 2.2 s would be split between T_2 and T_3, and if neither part exceeds the threshold, the attack passes undetected even though it was in progress the whole time.

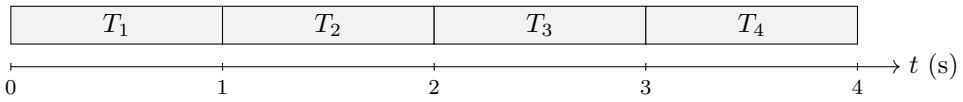


Figure 2: Tumbling windows of 1 s. Consecutive windows share a boundary but do not overlap.

Sliding windows [15] address this by letting consecutive windows overlap (Figure 3). The same attack would still be split between S_3 and S_5, but it lies entirely within S_4, which covers the interval from 1.5 s to 2.5 s. Two parameters are required instead of one: the window size, which determines the interval each window covers, and the slide interval, which determines how often a new window starts. The number of simultaneously active windows follows from the two as the window size divided by the slide interval. Because every point in time is covered by several windows at different offsets, any interval no longer than the window size minus the slide interval is fully contained in at least one window, so a short burst can no longer be split across a boundary.

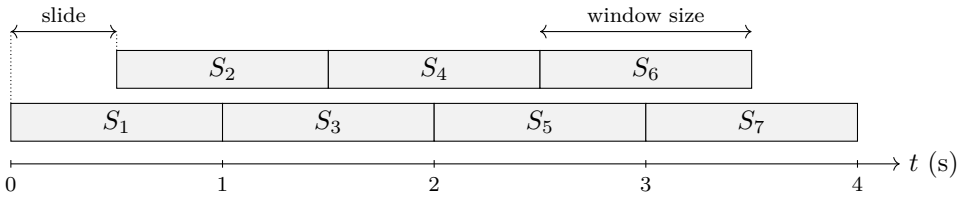


Figure 3: Sliding windows of 1 s with a slide interval of 0.5 s. Consecutive windows overlap by half their length.

The overlap is not free. When each window is maintained separately, a packet has to be registered in every active window, so the per-packet cost and the memory held by open windows grow in proportion to their number. A further consideration is that a window cannot be evaluated until it has closed. The slide interval therefore determines how often a decision is made, and thus how long an attack can be in progress before it is next evaluated, while the window size determines how much traffic each decision is based on. The choice of the two parameters is a trade-off between detection latency, resource consumption, and the amount of traffic each decision is based on.

The cost of overlap can be largely avoided when the computed statistic can be combined across intervals. The stream is then divided into short, non-overlapping sub-windows, called panes, whose length is the greatest common divisor of the window size and the slide interval. A partial result is computed once per pane, and each sliding window is obtained by combining the partial results of the panes it spans, so that every packet is processed only once [8]. This works directly for bounded aggregates such as counts and sums, whereas aggregates such as heavy hitters cannot in general be combined from constant-size partial results.

3 Methodology

This chapter describes how the two implementations were compared. Section 3.1 motivates the choice of a userspace baseline and a kernel-offloaded implementation of the same detector, and Section 3.2 describes the experiments: the hardware testbeds, the dataset, how its traffic was replayed, and which performance and detection metrics were collected.

3.1 Motivating Model Architecture

To investigate the effect of moving per-packet processing into the kernel, two implementations of the same detection model were developed. The first is a user-space baseline written in Go, in which packets are delivered to user space through a packet socket and counted there. The second is a kernel-offloaded implementation, in which per-packet counting and heavy-hitter tracking are performed by an XDP program written in C, while the detection stage remains in user space. Both implementations use the same HeavyKeeper variant and dimensions, address representation, windowing configuration and IQR-based detector. Some implementation details differ between the two versions, including hashing, fingerprint width, candidate management and concurrency, as described in Chapter 4. These differences are taken into account when interpreting the comparison.

Both implementations use HeavyKeeper as the basis for packet analysis, together with exact per-window packet and byte counters. Wickman and Rygaard [6] additionally used a Count-Min Sketch, but it was left out of this project. A Count-Min Sketch can estimate the count of an address it is asked about, but since it stores no addresses, it cannot tell which addresses are the heaviest (Section 2.3). HeavyKeeper, together with its top-K candidate set, provides both the heaviest addresses and an estimate of their counts, which is all the detector needs. A Count-Min Sketch would therefore only add per-packet work and memory, which matters in particular inside an eBPF program.

3.2 Experiment Design

The experiment compares the user-space baseline and the kernel-offloaded implementation on two testbeds, a Raspberry Pi 400 and a FABRIC virtual machine (Section 3.2.1), with traffic replayed from the CIC-DDoS2019 dataset (Section 3.2.2). Two kinds of data are collected: system performance, measured with blocking enabled and disabled, and detection quality, measured with blocking disabled, as described in Section 3.3.3.

3.2.1 Hardware Testbeds

The two implementations are evaluated on two testbeds with different capacity: a Raspberry Pi 400, representing a resource-constrained device such as a small-office router or an IoT gateway, and a virtual machine on FABRIC, representing server-class hardware. On both, one machine replays the traffic and the detector runs on a second machine that receives it. Table 3 summarizes the setups.

Raspberry Pi The detector runs on a Raspberry Pi 400, connected through a switch to a separate machine that replays the traffic. Since the Pi's Ethernet driver does not support native XDP, the XDP programs run in generic mode, which removes part of the advantage of processing packets in XDP (Section 2.2).

FABRIC FABRIC is a research infrastructure in which users reserve virtual machines at sites in the United States and Europe and connect them with dedicated network services [12]. A set of reserved resources is called a slice. The slice used here consists of two virtual machines at the same site, a sender and a detector, connected by a layer-2 bridge. The detector's network interface is a virtual function of a shared network card, whose driver supports native XDP, but for this project only generic XDP is tested.

Table 3: Testbed configurations. Raspberry Pi specifications from [3]; FABRIC VM configuration as described in [2].

	Raspberry Pi 400	**FABRIC VM**
CPU	Cortex-A72, 4 CPU, 1.8 GHz	AMD EPYC 7543 (x86-64), 2 vCPU, 2.8 GHz
Memory	4 GB	8 GB
Network	1 Gbps Ethernet, bcmgenet (v5)	Shared NIC, virtual function (mlx5)
OS and kernel	Ubuntu 24.04, Linux 6.8	Ubuntu 24.04, Linux 6.8
XDP mode	Generic	Generic
Traffic sender	Bare-metal OS (Ubuntu 24.04)	VM at the same site
Connection	Switch, direct ethernet cable connection	layer-2 bridge

3.2.2 Dataset

The traffic used in the evaluation comes from CIC-DDoS2019, a public dataset from the Canadian Institute for Cybersecurity [13], recorded over two days in a testbed where benign background traffic is mixed with a series of DDoS attacks. The attacks include reflection-based attacks, such as DNS, NTP and LDAP amplification, as well as direct floods, such as SYN and UDP floods.

The dataset is provided both as packet captures and as flow records extracted with CICFlowMeter. Each flow record includes a timestamp, a duration and a label identifying the flow as benign or naming its attack type. The packet captures provide the traffic for replay, while the flow labels provide the ground truth for evaluating detection quality.

CIC-DDoS2019 was chosen because it contains labelled volumetric attacks of several types and was used by Koshovyi and Wu [7], allowing a loose comparison with their results.

3.2.3 Traffic Replay

The traffic in the datasets was replayed using `tcpreplay`, an open-source tool that sends previously captured network traffic from PCAP files out onto a live network interface. It allows the replay rate to be controlled, either by preserving the original packet timing, scaling it by a multiplier, or sending at a fixed rate (in packets or megabits per second), which makes it possible to reproduce realistic traffic patterns in a controlled and repeatable way. Before replaying, the destination addresses in the datasets were rewritten to the IP address of the victim machine using `tcprewrite`, a companion tool in the same suite for editing packets in PCAP files. The specific configurations and commands used are listed in Appendix C.

3.3 Experiment Procedure

For the experiments, an approximately 40-minute excerpt was selected from CIC-DDoS2019 and replayed onto each testbed to compare the userspace baseline with the kernel-offloaded implementation. The main comparison used blocking-disabled runs to measure resource use and detection quality under sustained traffic load, complemented by runs with blocking enabled. This section describes the selected excerpt, the test configurations and execution, the measurements collected, and how detector verdicts were evaluated against the dataset labels.

3.3.1 Selected traffic excerpt

The selected excerpt spans 1 December 2018 from 14:21:52 to 15:01:51 UTC and contains approximately 78.6 million IP packets. It begins with approximately 14 minutes of benign traffic, followed by a high-rate flood and a period of lower-rate traffic, as seen in Figure 4. The dataset's flow labels identify one

attack type within the excerpt, DrDoS_DNS, and label the lower-rate traffic after the flood as attack traffic until the end of the excerpt. All times in this report are expressed in UTC.

The excerpt was chosen because it contains different traffic loads within a relatively short period. The initial benign period gives the detector time to establish a history of normal traffic before the attack begins. The same excerpt was used for both implementations and both blocking settings on each testbed.

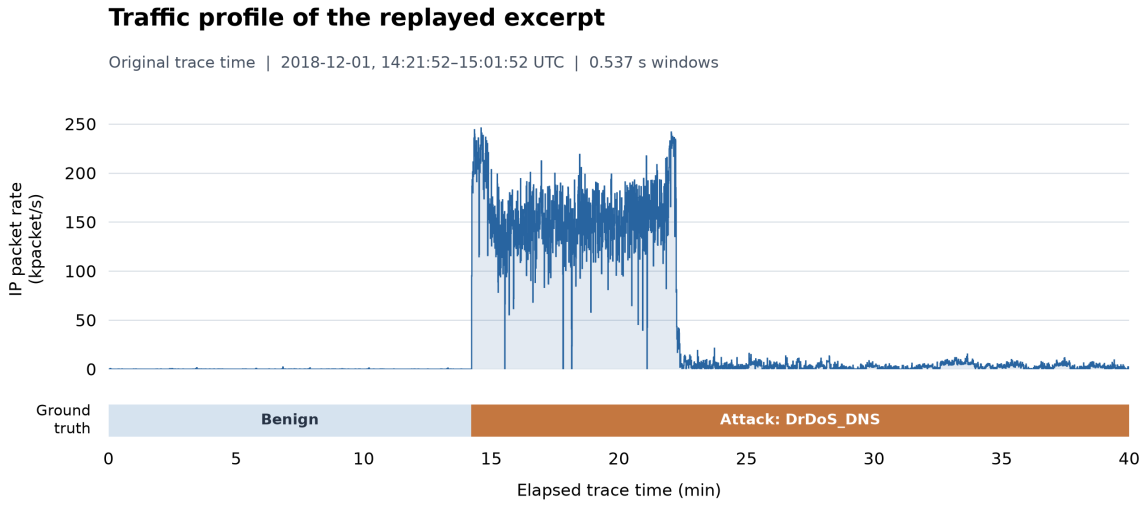


Figure 4: Packet rate of the selected excerpt, computed from the packet capture. The lower strip shows benign and attack periods based on the dataset's flow labels.

3.3.2 Test Configurations and Execution

Each implementation was tested with blocking disabled and enabled on both the Raspberry Pi 400 and the FABRIC VM, giving eight runs in total. Each configuration was run once. All runs used XDP in generic mode.

With blocking disabled, sources identified as malicious continued to send traffic through the detection pipeline. These runs formed the main comparison of resource use, packet loss and detection quality under sustained load. The blocking-enabled runs evaluated resource use when XDP dropped subsequent packets from blocklisted sources.

For each run, the detector was configured to run for 44 minutes and 30 seconds. Once the detector had started, traffic replay was initiated using the excerpt described in Section 3.3.1 and the replay procedure in Section 3.2.3, with one exception described in Section 5.1.3.

3.3.3 Data Collection

The measurements were divided into two categories: system performance, covering resource use and packet processing, and detection quality, covering how well the detector's verdicts matched the dataset labels.

System performance CPU utilization, packet loss and decision delay were recorded by the detector itself in all runs. Memory was recorded with a separate monitoring tool that was only used in the blocking-enabled runs.

- **CPU utilization:** whole-host CPU utilization, including software interrupts, read from `/proc/stat`. Detector-process CPU time was also recorded, but excludes the XDP work performed in the kernel. CPU utilization is reported both for the whole run and for the flood interval of each run, determined from its own traffic, independently of the detector's verdicts and the dataset labels: the packets counted per window were smoothed with a centred median over nine windows, and the flood interval spans from the first to the last window whose smoothed count exceeds 10,000 packets. In blocking-enabled runs, blocked packets and packets dropped by the capture socket were included, so that the flood remains visible when its packets are dropped.
- **Memory usage:** memory charged to the detector's cgroup, sampled every 0.5 s in the blocking-enabled runs. Separately charged BPF map memory was added to obtain the combined footprint.
- **Packet loss:** interface, capture-socket and detector counters were compared to account for interface drops, socket drops and packets discarded because their window had already closed. These losses were distinguished from intentional XDP blocking.
- **Decision delay:** the time from when a window ends until its verdicts are available.

Detection quality Detection quality was measured with blocking disabled, so that blocking decisions did not alter the traffic subsequently available for analysis.

- **Accuracy, precision, recall, F1 and false-positive rate:** calculated by comparing window-level verdicts with ground truth derived from the dataset's flow labels. Verdicts on individual IP addresses were not included in the scoring.

3.3.4 Detection Evaluation and Temporal Alignment

The dataset's flow labels were converted into time intervals using each flow's start timestamp and duration. Overlapping attack-labelled intervals were merged. An evaluated span was labelled malicious if it overlapped any attack interval, and benign otherwise. Since the detector evaluates each window together with its predecessor, the ground truth was determined over the same two-window span.

Live windows use timestamps from the detector machine and therefore need to be mapped to the dataset timeline before comparison with the labels. A constant time offset was initially estimated by matching packet counts per window against an offline reference run of the same excerpt. Where the automatically estimated onset of the high-rate flood deviated clearly from its onset in the recorded packet counts, the offset was instead set manually so that the two coincided, independently of the detector's verdicts. The saved verdicts were then re-evaluated using the adjusted timestamps. Aligning the flood onset does not, however, guarantee alignment throughout the replay.

Only windows whose complete evaluated span lay within the excerpt were eligible for scoring. The first 75 eligible, evaluated windows were excluded as initial scoring exclusion. The remaining window-level verdicts were compared with the ground truth to calculate the detection quality metrics described in Section 3.3.3.

4 Implementation

This section describes the two implementations compared in the evaluation: a userspace baseline (Section 4.1) and a kernel-offloaded implementation (Section 4.2). Both follow the same detection pipeline. Source IPs are counted per time window with HeavyKeeper, each window is evaluated by the IQR-based detector, and IPs judged malicious are added to a blocklist that an XDP program enforces in the kernel. The two differ in where the counting takes place. In the baseline, the XDP program only enforces the blocklist, and every packet that passes it is copied to userspace, where it is read, assigned to a window and counted (Figure 5). In the kernel-offloaded implementation, the counting is moved into the XDP program itself, which updates the window totals, the HeavyKeeper sketch and the top-K table in BPF maps as each packet arrives (Figure 7). Userspace then reads only a summary of each window from the maps once the window has ended, rather than every individual packet. The detection and blocking stage is identical in both implementations, and both evaluate each window together with the preceding one (Section 4.3), so the detector is given comparable values in either case. The remaining differences, which follow from the constraints of the XDP program, are described in Section 4.2.

4.1 Baseline Implementation

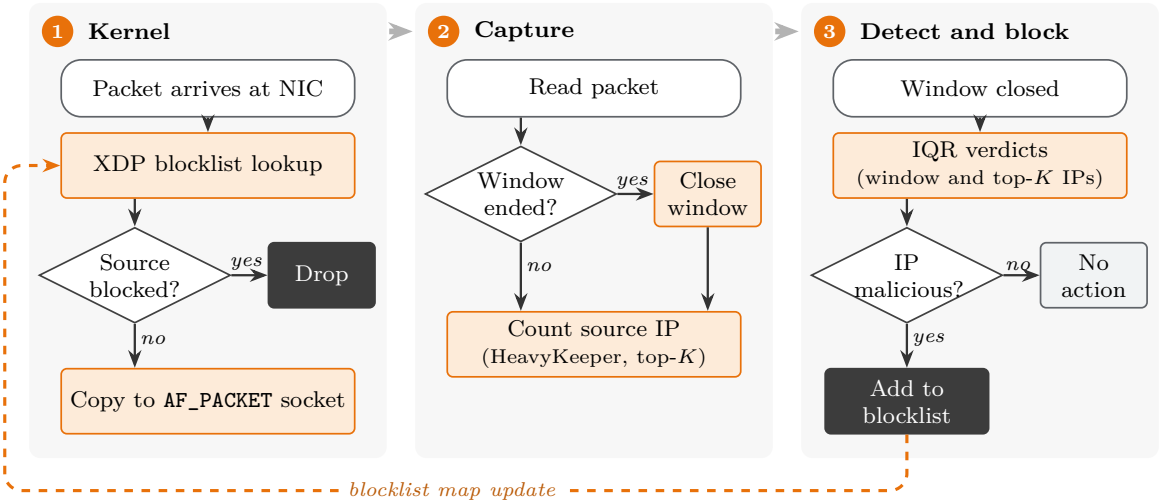


Figure 5: Packet path in the user-space baseline, from capture to blocking.

The user space implementation serves as the project's baseline, against which the kernel-offloaded implementation is compared. Figure 5 gives an overview of the pipeline, from packet capture to the blocking of malicious source IPs. It is divided into three stages: kernel, capture, and detection and blocking. Apart from the XDP blocklist, all of our processing takes place in userspace.

When a packet arrives at the NIC, the XDP program first looks up its source IP in the blocklist. If the address is present, the packet is dropped immediately. Otherwise, it continues up the network stack, and a copy is delivered to an `AF_PACKET` socket. This is a raw socket that gives userspace programs access to packets at the link layer.

In the capture stage, the userspace program reads the captured packets one at a time. For each packet, it first checks whether the current time window has ended (Section 4.3), and if so, closes it. The packet's source IP is then counted using HeavyKeeper, which tracks the top-K most frequent

sources (Section 4.1.2).

In the detection and blocking stage, which runs each time a window is closed, an IQR-based detector (Section 4.1.3) produces verdicts both for the window as a whole and for each of the top-K IPs. Any IP judged malicious is added to the XDP blocklist map, so that subsequent packets from that source are dropped in the kernel. The IP stays blocked until it has not been flagged for a set period.

4.1.1 Flow

Every component in the pipeline identifies traffic by its source address. To ensure that the sketch, the detector and the kernel implementation all identify an address in the same way, the address representation is defined once, in a shared flow package, together with the hash function used by the baseline. The kernel-offloaded implementation uses the same representation but its own hash function (Section 4.2.2)

Key A source address is represented by a fixed-size key:

```
type Key [16]byte
```

IPv6 addresses are stored as they are, and IPv4 addresses are stored in IPv4-mapped form (`::ffff:a.b.c.d`), so both address families share one representation. Since the key is a plain 16-byte array, it can be copied without allocation, used directly as a map key, and mirrored as the key of a BPF map in the kernel implementation. Addresses are converted to keys directly from the bytes of the IP header, so no string representation is created in the packet path.

Hashing The key is reduced to a 32-bit hash by folding it one 32-bit word k_i at a time and mixing after each word:

$$h_0 = \mathrm{mix}(k_0), \qquad h_i = \mathrm{mix}(h_{i-1} \oplus k_i), \quad i = 1, 2, 3, \tag{3}$$

with the final hash $h = h_3$. The mixing function is an integer hash with good avalanche properties, which is needed since addresses cluster heavily in their low bits. It is also a bijection on 32-bit integers, so every step is a bijection in the word it absorbs. Since all IPv4-mapped keys share their first three words, two different IPv4 addresses can therefore never produce the same hash. For IPv6, the result is an ordinary 32-bit hash. The hash is computed once per packet, and the components that need several hash values, such as the rows of the HeavyKeeper sketch, derive them from this value rather than from the key itself.

4.1.2 HeavyKeeper

The HeavyKeeper sketch [4] identifies the source addresses responsible for the largest share of the traffic within a time window, and estimates how many packets each of them sent. Its interface consists of `Insert`, which records one packet and returns the resulting estimate, `Query`, which returns an estimate without modifying the sketch, and `Reset`, which clears it for reuse in a later window.

The implementation follows the Hardware Parallel Version of Yang et al. [4], in which every mapped bucket whose fingerprint does not match is decayed independently of the others. The alternative, the Software Minimum Version, decays only the smallest of the mapped counters and achieves a higher accuracy, but gives up this independence, since the rows must be read, compared and written as one dependent operation. The parallel version was chosen because it maps directly onto the kernel implementation. The selective increment optimization of [4] is not implemented: a bucket with a matching fingerprint is always incremented.

Bucket layout Each bucket holds a fingerprint and a counter:

```
type Bucket struct {
        Fingerprint uint32
        Count       uint32
}
```

Both fields are 32 bits, giving eight bytes with no padding and eight buckets per 64-byte cache line. With d rows of w buckets, the sketch occupies

$$M = d \cdot w \cdot 8 \text{ bytes}, \tag{4}$$

which with the default dimensions $d = 4$ and $w = 1024$ amounts to 32 KiB according to Equation 4. A 16-bit counter would saturate at 65 535 packets, which a single source can exceed within one window at the packet rates of a volumetric flood. The counter therefore saturates at its 32-bit maximum rather than wrapping, since a wrapped counter would read as an empty bucket and the flow holding it would be evicted by the next packet. The fingerprint is widened to match the counter, since the space would otherwise be padding, and a wider fingerprint makes collisions rarer than with the 16-bit fingerprints used as an example in [4].

Hashing A flow is identified by the 16-byte key described in Section 4.1.1, which is reduced to a 32-bit hash h of Equation 3 once per packet. Both the row indices and the fingerprint are derived from h rather than from the key itself, so the key is read only once. With a bijective mixing function mix, a row-specific seed s_j and a separate salt s_{fp}, the index in row j and the fingerprint are

$$i_j = \text{mix}(h \oplus s_j) \bmod w, \qquad F = \text{mix}(h \oplus s_{\text{fp}}), \tag{5}$$

where the modulo is computed as a bit mask, which requires w to be a power of two. Since mix is a bijection, two flows share a fingerprint in Equation 5 exactly when their key hashes collide. For IPv4 this cannot happen, since the folding step that absorbs the address bytes is itself a bijection and IPv4-mapped keys agree in their preceding words. Fingerprint collisions are therefore an IPv6-only concern here.

Decay When a mapped bucket is held by another flow, its counter C is decremented with probability

$$P_{\text{decay}} = b^{-C}, \tag{6}$$

where the decay base is set to $b = 1.08$, the value used in [4]. To avoid floating-point operations at run time, the probabilities of Equation 6 are precomputed into a table of 64 entries scaled to the range of a 32-bit integer,

$$T[C] = \left\lfloor b^{-C} \cdot (2^{32} - 1) \right\rfloor, \qquad 0 \le C < 64, \tag{7}$$

so that the decision is a comparison between a 32-bit random number and $T[C]$ of Equation 7. Counters of 64 or above are treated as having probability zero, in line with the observation in [4] that the probability is close to zero once the counter is large; at $b = 1.08$, b^{-64} is already below 0.008. The random numbers come from a seeded SplitMix64 generator embedded in the sketch, so a run is reproducible from its initial seed.

Top-K candidates Because the sketch stores fingerprints rather than addresses, it can estimate the size of a flow presented to it but cannot enumerate the largest flows on its own. A separate candidate set therefore holds the actual key and estimate of the k largest flows seen so far in the window, $k = 10$ by default. Since k is small, the set is implemented as a flat slice with a map from key to position, and the minimum is found by a linear scan rather than a heap.

A flow whose estimate is zero is held in no bucket and is a mouse flow, so it is never offered to the candidate set. Until the set is full, every offered flow is admitted. After that, a flow that is not already a candidate replaces the current minimum $n_{\min}$ only if its estimate is exactly $n_{\min}+1$. As shown in [4], a larger estimate can only result from a fingerprint collision, and admitting such a flow would let an overestimated mouse flow occupy the candidate set.

A sketch and its candidate set belong to one time window and are driven by a single goroutine, so neither is synchronized. Both are reset and reused between windows rather than reallocated.

4.1.3 IQR

The IQR model, based on the statistical baseline of Wickman and Rygaard [6], consists of two parts: the history and the detector. A history stores recent samples of normal traffic and computes a threshold from them. The detector holds several histories, compares each observed packet count against the threshold of the appropriate history, and decides whether the sample is added to it. Detection happens at two levels, the aggregate traffic of each window and the traffic of the individual IPs within it, so three kinds of histories are kept: one for the window totals, one for each tracked IP, and one shared by all IPs.

Each history calculates its threshold based on the 25th (Q_1) and 75th (Q_3) percentiles of its samples, applying linear interpolation and the IQR from Equation 1:

$$\text{Base} = \max(Q_3 + \text{IQRMultiplier} \times \text{IQR},\ \text{BaseThreshold}) \tag{8}$$

$$\text{Final Threshold} = \text{Base} + \text{OffsetMultiplier} \times \sigma \tag{9}$$

Here, σ represents the sample standard deviation ($n-1$ denominator), which accounts for the history containing a limited sample of normal traffic rather than the entire population. The base threshold (Equation 8) prevents the minimum threshold from collapsing to zero during periods of low activity, while the σ term (Equation 9) allows the model to absorb normal traffic variance, reducing false positives during benign bursts.

The parameters are configurable and default to the values established by Wickman and Rygaard: a base threshold of 200, an IQR multiplier of 1.5, and an offset multiplier of 2. Consequently, a count of at most $200 + 2\sigma$ packets is never classified as anomalous. As noted in Section 6.3, these defaults have not been explicitly calibrated for our traffic.

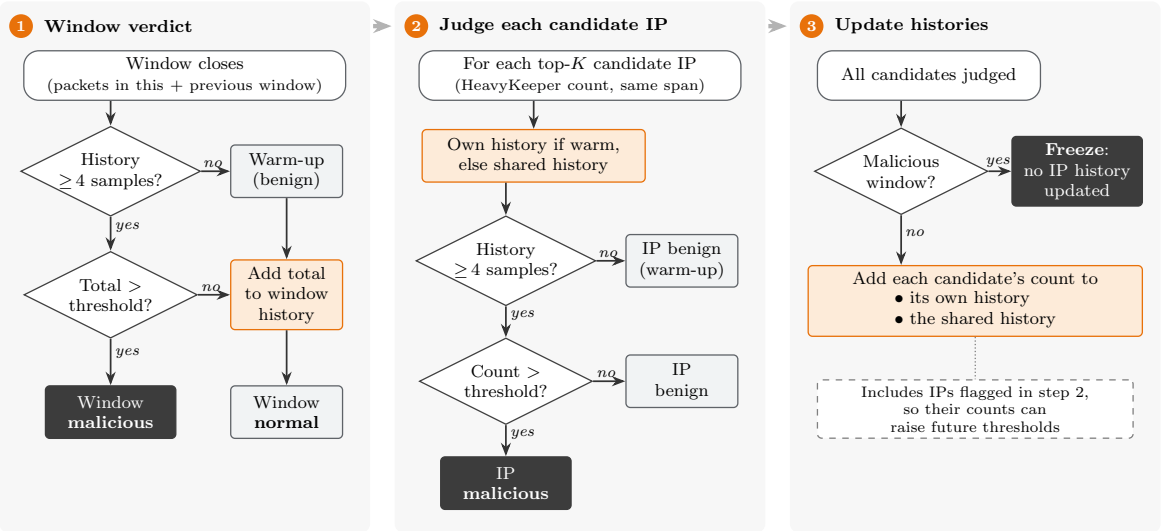


Figure 6: Evaluation performed by the IQR detector when a window closes. (1) The window total is judged against the window history. (2) Every top-K candidate is judged against its own history once that history is warm, and against the shared history until then. (3) If the window was normal, the counts are added to the IP histories.

Figure 6 shows the evaluation performed when a window closes. The values judged are the combined counts produced by the windowing stage (Section 4.3, Equations 12 and 13), and the histories are built from the same combined values.

Every history has a warm-up phase, since a meaningful threshold cannot be computed without prior data. Until a history holds four samples (configurable), every evaluation against it yields a benign verdict. Warm-up applies separately to the window history, each IP's history, and the shared history.

In step 1, the detector compares the window's total packet count against the window-level threshold. If the window is normal, its total is added to the window history; if it is malicious, the history is left unchanged so that the attack does not raise its own threshold.

In step 2, the detector judges the top-K candidates identified by HeavyKeeper, using their estimated packet counts. This is done in every window, normal or malicious, since a malicious window is exactly when the attacking IPs need to be identified. An IP is judged against its own history once that history has left warm-up, and against the shared history, built from the counts of all candidates in normal windows, until then. Only if both histories are still warming up is the IP treated as benign. Since histories only learn from normal windows, the own history of an IP first observed during an attack would not fill for as long as the attack lasts; judging it against the shared history lets it be flagged immediately. The cost is that the shared history describes the typical candidate, so a legitimate source much heavier than the usual candidates, such as a NAT gateway, can be flagged until its own history has left warm-up.

In step 3, the IP histories are updated, unless the window was malicious, in which case neither the individual nor the shared histories are updated, which prevents a sustained attack from poisoning the baselines. In a normal window, each candidate's count is added to both its own history and the shared history, including candidates flagged in step 2, whose counts can therefore raise both their own future threshold and the shared threshold used for new IPs. A candidate's own history is updated as soon as it has been judged, which cannot affect any other verdict since each IP appears at most once among the candidates. The shared history, against which every new IP is judged, is updated only once all

candidates have been judged, so the order of evaluation does not affect any verdict.

Each history is a bounded FIFO buffer that evicts its oldest sample when full. The window history and each per-IP history keep at most 500 samples, and the shared history at most 5000. At most 1000 IPs are tracked at a time, with least-recently-used (LRU) eviction, so per-IP state is bounded at 1000×500 samples (at most 4 MB) regardless of how many distinct source addresses are observed. This matters for attacks that use rotating or spoofed source addresses. An evicted IP that returns starts over with an empty history and is judged against the shared history until its own history has left warm-up again.

4.2 Kernel-Offloaded Implementation

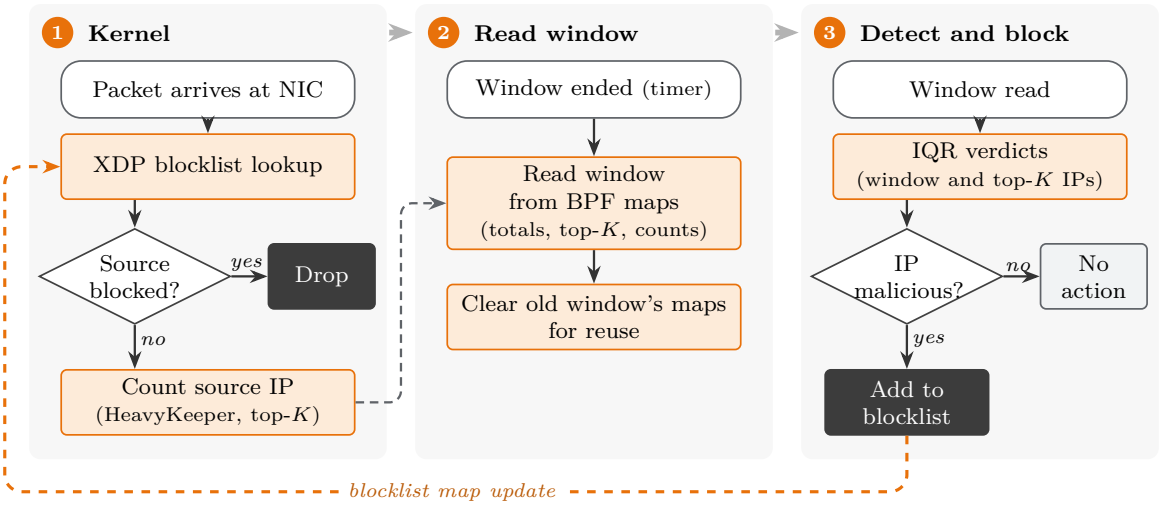


Figure 7: Packet path in the kernel-offloaded implementation, from capture to blocking.

Figure 7 shows the kernel-offloaded pipeline. It has the same three stages as the baseline, except that the capture stage is replaced by a stage that reads each completed window from the BPF maps. The detection and blocking stage is unchanged from Section 4.1.

In the kernel stage, the XDP program parses the source address of each packet into the same 16-byte key as the baseline and looks it up in the blocklist. A blocked packet is dropped. Any other packet is counted in the slot of the current window and passed on to the network stack, without any copy being made for user space. In the read-window stage, user space reads the totals and top-K candidates of each completed window from its slot, as described in Section 4.3.3.

4.2.1 BPF Maps

Table 4 lists the maps shared by the XDP program and userspace. Each slot has its own sketch and top-K map, so that userspace can read and clear one slot while XDP writes to another. The window totals are kept in per-CPU arrays, which avoid contention between CPUs and are summed by userspace when a window is read (Section 2.1). The blocklist has the same layout as the baseline's, keyed by the 16-byte address.

Table 4: BPF maps of the kernel-offloaded implementation.

Map	Type	Entries	Value
`sketch_0`–`sketch_3`	Array	$d \cdot w = 4096$	HeavyKeeper bucket
`topk_map_0`–`topk_map_3`	Array	10	Address and estimate
`slot_stats`	Per-CPU array	4	Packets and bytes per slot
`blocked_stats`	Per-CPU array	4	Dropped packets and bytes per slot
`blocklist_map`	Per-CPU hash	1000	Time of last drop, drop count

4.2.2 HeavyKeeper

The XDP program implements the same variant of HeavyKeeper as the baseline, the Hardware Parallel Version, with the same dimensions (Section 4.1.2). This section describes only what differs, as a consequence of the eBPF verifier and of the program running on several CPUs in parallel.

Bucket layout A bucket holds a 16-bit fingerprint, 16 bits of padding and a 32-bit counter, eight bytes as in the baseline. A bucket whose counter is zero is empty, and the fingerprint 0 is reserved for empty buckets, so a fingerprint that hashes to 0 is replaced by 1. The $d \cdot w$ buckets of a slot are stored in its sketch map (Table 4) at index $j \cdot w + i$ for row j and column i.

Hashing Each row index is computed with MurmurHash3 over the full 16-byte key, with a row-specific seed, and the fingerprint with a further hash under a separate seed, folded to 16 bits by combining its two halves with XOR. A packet thus requires five hashes, whereas the baseline hashes the key once and derives the row indices and the fingerprint from that value (Equation 5).

Decay The decay probabilities are precomputed into a table of 64 entries, as in the baseline (Equation 7), but scaled to 16 bits:

$$T_{16}[C] = \left\lfloor b^{-C} \cdot (2^{16} - 1) \right\rfloor, \qquad 0 \le C < 64. \tag{10}$$

A counter is decayed if the lower 16 bits of a random number from the `bpf_get_prandom_u32` helper are smaller than $T_{16}[C]$ in Equation 10, and counters of 64 or above are never decayed, as in the baseline. No floating-point operations are needed (Section 2.1). Since the random numbers are not seeded, runs cannot be reproduced exactly.

Verifier constraints The loops over the rows and the top-K entries are unrolled at compile time, and all helper functions are inlined. Every map lookup is followed by a null check, as required by the verifier. Since the map a program accesses must be known when it is verified, the sketch and top-K map of the active slot are selected with an explicit branch per slot rather than by an index.

Concurrency The sketch is shared by all CPUs that receive packets (Section 2.1). Incrementing a matching counter is atomic, but claiming an empty bucket, decaying a counter and updating the top-K set are not, so an update can occasionally be lost when two CPUs access the same bucket at once. This is accepted, since the sketch only provides an estimate. The maps updated on every packet regardless of the sketch use per-CPU variants instead (Section 4.2.1).

Top-K candidates Each slot has K entries, each holding an address and its estimate. An address already in the set has its estimate updated, an empty entry is filled directly, and otherwise the entry with the smallest estimate is replaced if the new estimate is larger. Unlike the baseline, the program does not require the estimate to be exactly one above the minimum, so estimates inflated by fingerprint collisions are not filtered out.

4.3 Window mechanism

The window mechanism groups incoming packets into fixed-length tumbling windows and produces the values that the detector evaluates when a window closes. Following the pane approach described in Section 2.5 [8], each window is combined with the window immediately before it, which gives the temporal resolution of a sliding window with 50% overlap while every packet is inserted into only one window. The epochs and the combined values are common to both implementations, whereas the way windows are stored and closed differs between user space and kernel space.

4.3.1 Common design

Epochs Time is divided into epochs, numbered by the number of whole window lengths since a fixed reference time. A packet with timestamp t (in nanoseconds) belongs to epoch

$$e = \left\lfloor \frac{t}{L} \right\rfloor, \tag{11}$$

and the window of epoch e covers the half-open interval $[eL, (e+1)L)$, so every instant falls in exactly one window. The window length is $L = 2^{29}\,\text{ns} \approx 536.9\,\text{ms}$. A power of two is used because the kernel-space implementation computes the same formula with a bit shift instead of a division, and both implementations use the same length.

Combined values When the window for epoch e closes, the combined packet count of the window and its predecessor is formed as

$$P_e^{\text{comb}} = P_e + P_{e-1}. \tag{12}$$

As discussed in Section 2.5, two HeavyKeeper sketches cannot be merged into one. The per-IP counts are therefore combined per candidate: for every address x in the candidate set, its estimates from both windows' sketches are added,

$$\hat{c}_e^{\text{comb}}(x) = \hat{c}_e(x) + \hat{c}_{e-1}(x). \tag{13}$$

Keeping the previous window's sketch, and not only its candidate list, means that an address that only becomes a candidate in the current window still has its packets from the previous window counted. A window without an immediately preceding window, such as the first window of a run, cannot be combined and is not evaluated.

4.3.2 User space

In user space, each window holds exact totals for its interval, together with its own HeavyKeeper sketch and top-K candidate set:

```
type Window struct {
        Epoch      epoch.Epoch
        Start, End time.Time
        Packets    uint64
        Bytes      uint64
        hk         *heavykeeper.HeavyKeeper
        topK       *heavykeeper.TopK
}
```

The timestamp t is that of the packet: the trace's own timestamp when reading a trace file, and the kernel's arrival timestamp during live capture. For every packet, the epoch is computed from its timestamp, `Packets` and `Bytes` are incremented, the source address is inserted into the sketch, and the candidate set is updated with the returned estimate. A zero estimate means that the address does not hold a bucket in the sketch, and it is then not considered a candidate. Only two windows are kept,

the current one and the previous one, which swap roles each time a window closes. Since all processing runs on a single goroutine, no synchronization between them is needed.

Windows are driven by packet timestamps: when a packet belongs to a later epoch than the current window, every window up to that epoch is closed first. When reading a trace file, the timestamps are those of the trace, so the windows are independent of how fast the file is read. During live capture, packets may stop arriving altogether, so the capture loop also closes the current window when its end time has passed. Packets whose timestamp falls in an already closed window are dropped and counted, since adding them to the current window would place them in an epoch they do not belong to.

Pauses in the traffic are handled differently for the two sources. During live capture, a quiet period is real data, and every empty window is closed. Trace files such as CIC-DDoS2019 are, however, split into many files with gaps between the recordings. To avoid filling the histories with zeros from these artifacts, at most ten consecutive empty windows are closed when reading a trace file, after which the aggregator jumps directly to the epoch of the next packet. Such a jump is followed by a window without a predecessor, which is therefore not evaluated.

4.3.3 Kernel space

In kernel space, the packets are counted by the XDP program, and user space only reads each window back once it has closed. The windows are allocated once, as a fixed pool of four slots in BPF maps, each holding a HeavyKeeper sketch, a top-K candidate set and the packet and byte totals. Which slot is active is not tracked explicitly: the XDP program reads the kernel's `CLOCK_TAI` and writes to the slot at index $e \bmod 4$ for the current epoch e of Equation 11. User space computes the same index from the same clock, so when time passes into a new epoch, packets are written to the next slot without any coordination between the two.

Windows are therefore driven by the clock rather than by packets. User space sleeps until a window has ended, plus a grace period of 10 ms that lets packets stamped just before the boundary finish updating the maps on other CPUs. When the window for epoch e closes, slots e and $e-1$ are read and combined, and slot $e+2$ is cleared, which the XDP program only starts writing a full window later. If user space falls so far behind that a slot is reused before it has been read, that window is discarded rather than evaluated with mixed counts.

5 Results

This section compares the kernel-offloaded implementation with the userspace baseline on the Raspberry Pi 400 and the FABRIC VM. The same approximately 40-minute traffic excerpt was used throughout, with an initial benign period followed by a high-rate flood and a lower-rate attack tail. The experimental procedure is described in Section 3.3.

Table 5 summarizes the metrics reported, and Table 6 lists the test configurations. Each configuration was run once. Runs with blocking disabled form the main comparison of system performance and detection quality. Additional runs with blocking enabled show resource use when detected sources are blocked.

System Performance	**Detection Quality**
Packet loss	Accuracy
Decision delay	Recall
CPU utilization	Precision
Memory usage	F1 score
	False-positive rate

Table 5: Main metrics reported across the experiments.

Testbed	Blocking	Implementation
Raspberry Pi 400	Disabled	Userspace
	Disabled	Kernel
	Enabled	Userspace
	Enabled	Kernel
FABRIC VM	Disabled	Userspace
	Disabled	Kernel
	Enabled	Userspace
	Enabled	Kernel

Table 6: Overview of the eight experimental runs. With blocking enabled, XDP drops packets from blocklisted sources.

The following subsections present the results with blocking disabled and enabled, respectively. Within each subsection, results are reported for the Raspberry Pi 400 and the FABRIC VM. Section 5.3 summarizes the findings.

5.1 Blocking Disabled

This subsection compares system performance and detection quality with blocking disabled. Keeping blocking disabled prevents the detector's blocking decisions from changing the traffic available for subsequent analysis. It also exposes both implementations to the replayed traffic throughout the high-rate attack phase, allowing resource use and packet loss to be compared under sustained load. Packets may still be lost before being counted, as examined in the following subsection.

5.1.1 Packet Accounting and Loss

Packets are followed from the network interface to the detector's windows. The kernel implementation has no capture socket and does not discard late packets, so these stages do not apply to it.

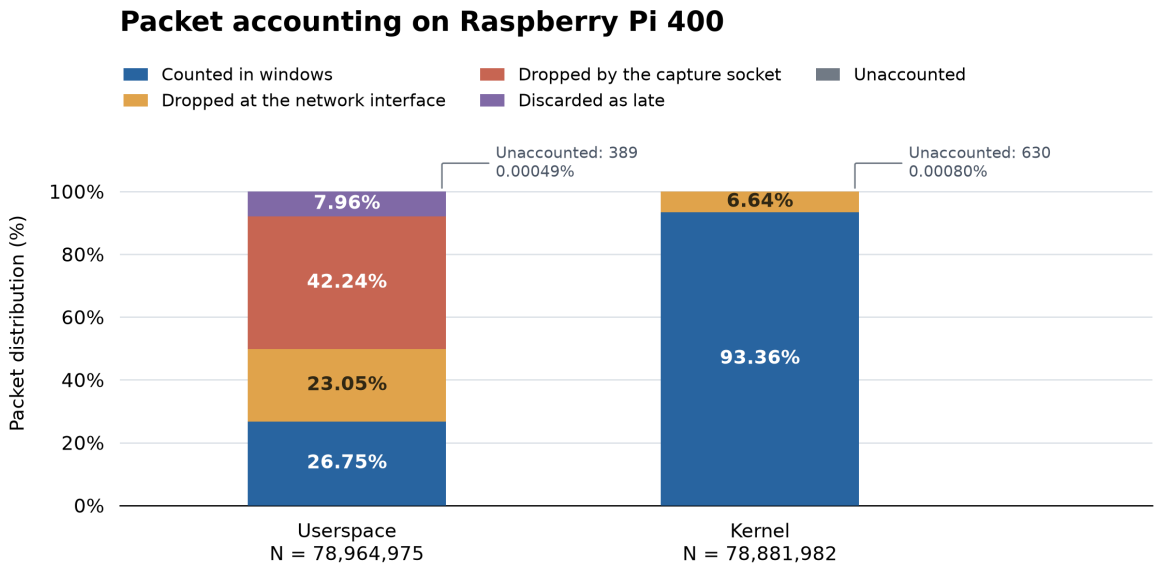


Figure 8: Packet accounting on the Raspberry Pi 400 with blocking disabled. Each bar shows where the packets received at the network interface (N) were counted or lost.

	Userspace	Kernel
Received at the network interface	78,964,975 (100%)	78,881,982 (100%)
Dropped at the network interface	18,201,335 (23.05%)	5,238,189 (6.64%)
Dropped by the capture socket	33,356,522 (42.24%)	–
Discarded as late	6,283,426 (7.96%)	–
Unaccounted	389 (0.00%)	630 (0.00%)
Counted in windows	21,123,303 (26.75%)	73,643,163 (93.36%)

Table 7: Packet accounting on the Raspberry Pi 400 with blocking disabled. Percentages are of the packets received at the network interface, including those dropped there. Interface drops are as reported by the kernel and may include packets dropped in the kernel's receive path, not only by the network card. The kernel-offloaded implementation has no capture socket and counts packets as they arrive, so the corresponding stages do not apply.

Raspberry Pi 400 On the Raspberry Pi, the userspace baseline counts 26.75% of the total packets received at the network interface, against 93.36% for the kernel implementation (Figure 8). In absolute terms, the kernel implementation counts approximately 3.5 times as many packets as the userspace baseline (Table 7). The userspace baseline loses packets at three stages: 23.05% at the network interface, 42.24% at the capture socket, and 7.96% as late packets that arrive after their window has closed. The kernel implementation loses packets only at the network interface, 6.64%.

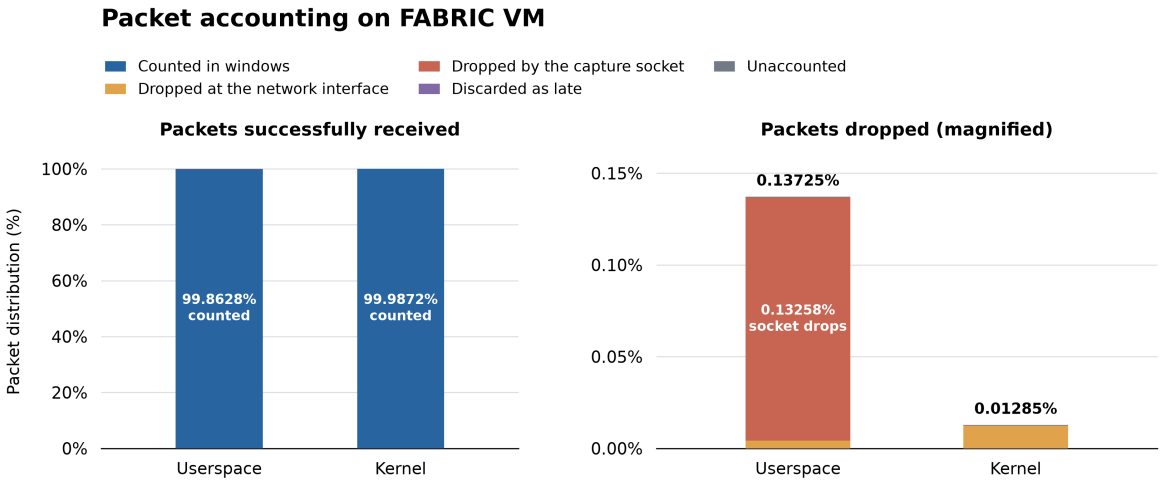


Figure 9: Packet accounting on the FABRIC VM with blocking disabled. The y-axis shows the packet distribution, split between the kernel and userspace implementations. The left bar graph shows the distribution of packets counted in windows. The right bar graph shows the distribution of packets dropped.

	Userspace		Kernel	
	Packets	%	Packets	%
Received at the network interface	78,563,556	100.000	78,563,556	100.000
Dropped at the network interface	3,251	0.004	9,689	0.012
Dropped by the capture socket	104,156	0.133	–	–
Discarded as late	13	0.000	–	–
Unaccounted	405	0.001	405	0.001
Counted in windows	78,455,731	99.863	78,553,462	99.987

Table 8: Packet accounting on the FABRIC VM with blocking disabled. Percentages are of the packets received at the network interface, including those dropped there. Interface drops are as reported by the kernel. The kernel-offloaded implementation has no capture socket and counts packets as they arrive, so the corresponding stages do not apply.

FABRIC VM On the FABRIC VM, both implementations count nearly all packets received at the network interface: 99.863% for the userspace baseline and 99.987% for the kernel implementation (Figure 9, Table 8), a difference of 0.124 percentage points against 66.61 on the Raspberry Pi. The userspace baseline loses 104,156 packets at the capture socket and 13 as late drops. Drops at the network interface are higher for the kernel implementation, 9,689 against 3,251, but it still counts 97,731 more packets in total. At this load, the capture-socket and late-drop losses that dominate on the Raspberry Pi are negligible, so the two detectors receive essentially the same traffic.

5.1.2 CPU Utilization

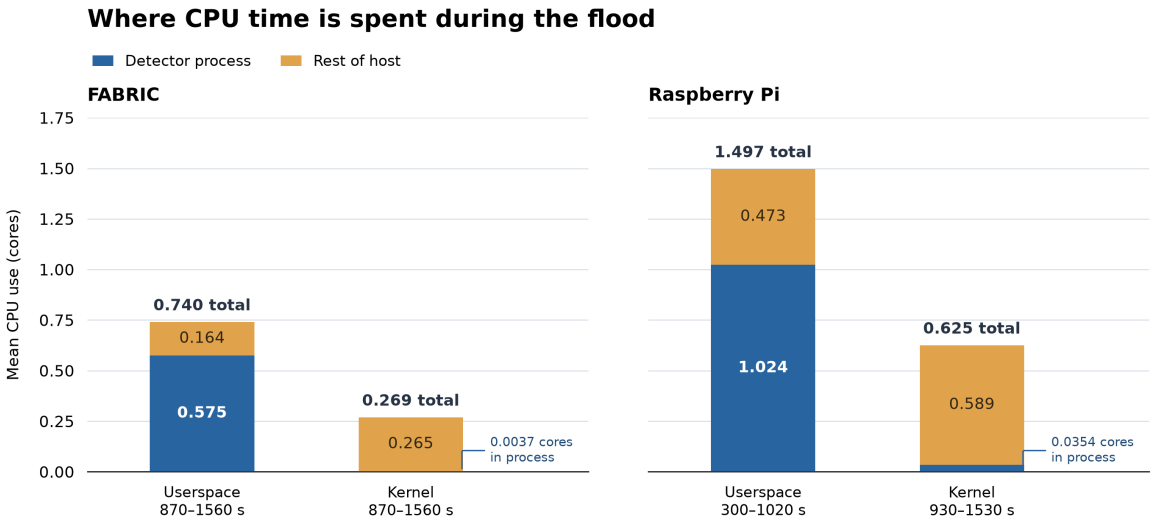


Figure 10: Victim CPU utilization with blocking disabled, including software-interrupt time. The FABRIC VM is displayed on the left and the Raspberry Pi 400 on the right. Both bar graphs are split between the userspace and kernel implementations. The y-axis shows the mean CPU use across all cores for the flood interval of the volumetric attack.

Raspberry Pi 400 On the Raspberry Pi, the flood interval of each run, determined as described in Section 3.3.3, is 272.7–1044.2 s after detector start for userspace and 904.1–1575.7 s for the kernel implementation. The intervals start at different times because the user-space detector was started later in the replay (Section 5.1.3). Figure 10 uses these same intervals.

	Userspace	Kernel	Run	Unit
CPU utilization, victim host	13.44	6.65	Whole run	% of all cores
CPU utilization, victim host	38.7	16.5	Flood interval	% of all cores
CPU utilization, detector process	101.6	3.5	Flood interval	% of one core
Counted rate, mean	24.3	104.9	Flood interval	kpps
Counted rate, maximum window	50.6	219.3	Flood interval	kpps
CPU time per counted packet, victim host	61.7	6.2	Flood interval	μs

Table 9: CPU utilization and counted packet rate on the Raspberry Pi 400 with blocking disabled. CPU utilization for victim host covers the whole machine, including the software-interrupt time where XDP runs, while detector process CPU covers only the detector's own process and can exceed 100% because Go uses several threads. Victim host CPU time per counted packet is the victim host CPU time during the flood divided by the packets counted in windows during the flood.

During the flood, host CPU utilization is 38.7% for userspace and 16.5% for the kernel implementation, while the kernel implementation counts 4.3 times as many packets per second (Table 9). Victim host CPU time per counted packet is therefore 61.7 versus 6.2 μs. The userspace detector process uses 101.6% of one core during the flood, the kernel detector process 3.5%.

FABRIC VM The FABRIC flood intervals are 854.2–1581.1 s for userspace and 855.2–1584.3 s for the kernel run, measured from the first recorded window and determined as described in Section 3.3.3. Figure 10 uses these same intervals.

	Userspace	Kernel	Run	Unit
CPU utilization, victim host	10.20	4.25	Whole run	% of all cores
CPU utilization, victim host	37.1	14.2	Flood interval	% of all cores
CPU utilization, detector process	56.1	0.43	Flood interval	% of one core
Counted rate, mean	103.6	103.4	Flood interval	kpps
Counted rate, maximum window	143.3	143.2	Flood interval	kpps
CPU time per counted packet, victim host	6.69	2.73	Flood interval	μs

Table 10: CPU utilization and counted packet rate on the FABRIC VM with blocking disabled. CPU utilization for victim host covers the whole machine, including the software-interrupt time where XDP runs, while detector process CPU covers only the detector's own process. Victim host CPU time per counted packet is the victim host CPU time during the flood divided by the packets counted in windows during the flood.

During the flood, the counted rates are almost equal, 103.6 versus 103.4 kpps, while host utilization is 37.1% for userspace and 14.2% for the kernel implementation. Victim host CPU time per counted packet is 6.69 versus 2.73 μs (Table 10). The detector process accounts for 56.1% of one core in userspace and 0.43% in the kernel-offloaded run. XDP's per-packet work is outside that process and remains part of host CPU.

5.1.3 Detection Quality

Detection quality is scored against window-level ground truth derived from the flow labels, excluding the first 75 evaluated windows as warm-up.

The automatic alignment reported low correlation and ambiguous peaks for all runs (Section 3.3.4). On the FABRIC VM, the estimated offsets placed the onset of the flood within 2 s of its observed onset and were kept. On the Raspberry Pi, they placed it approximately 160 s off, and the offsets of both runs were set manually. The flood lasted approximately 728 s in the FABRIC runs, compared with 489 s in the offline reference, which suggests that the replay ran slower than real time during the flood. Since a constant offset cannot correct for this, the alignment is accurate at the onset of the flood but less reliable later in the run, and the window-level metrics below are therefore approximate.

Raspberry Pi 400 Due to a mistake during the experiment, the user-space detector on the Raspberry Pi was started approximately 9.6 minutes after the replay had begun, rather than before it as intended (Section 6.3). This shortened its warm-up and left 432 benign windows for scoring, compared with 1,511 for the kernel-offloaded run. The detection quality of the two runs is therefore not directly comparable, whereas packet loss and CPU utilization during the flood are unaffected.

Figure 11 shows the window-level outcomes of both implementations over time, compared with the ground truth, and Table 11 summarizes the resulting metrics. The kernel-offloaded implementation achieves a higher recall (0.89 against 0.76) and a lower false-positive rate (2.25 against 2.78), while the user-space baseline has a slightly higher precision. The larger number of true negatives for the kernel-offloaded implementation mainly reflects its longer benign scoring period. The difference in recall does not depend on the manual alignment: with the automatically estimated offset, the recall of the user-space baseline was 76.5%. Of the 195 flood windows the baseline did not flag, 183 contained no counted packets at all in their two-window span, which indicates that most of its missed detections during the flood are a consequence of packet loss rather than of the detector itself (Section 5.1.1). The

shorter warm-up may also have contributed, but the two effects cannot be fully separated from these runs.

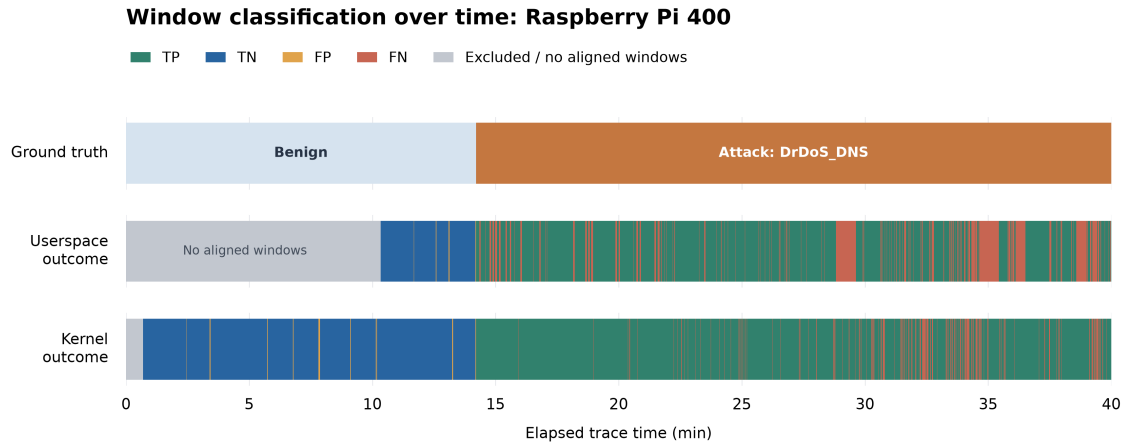


Figure 11: Window classification over time for the Raspberry Pi 400 with blocking disabled.

	Userspace	Kernel
Windows scored	3,314	4,393
Attack-labelled windows	2,882	2,882
Benign-labelled windows	432	1,511
True positives (TP)	2,191	2,577
False positives (FP)	12	34
True negatives (TN)	420	1,477
False negatives (FN)	691	305
Recall (%)	76.02	89.42
Precision (%)	99.46	98.70
False-positive rate (%)	2.78	2.25
F1 score	0.862	0.938
Accuracy (%)	78.79	92.28

Table 11: Window-level detection quality on the Raspberry Pi 400, with blocking disabled.

FABRIC VM On the FABRIC VM, the two implementations have similar detection quality: recall is 92.2% against 91.9%, precision 98.8% against 98.6%, and F1 0.954 against 0.951 for the user-space baseline and the kernel-offloaded implementation, respectively (Table 12). Both implementations counted nearly all packets on this testbed, and the small differences do not indicate an advantage for either implementation.

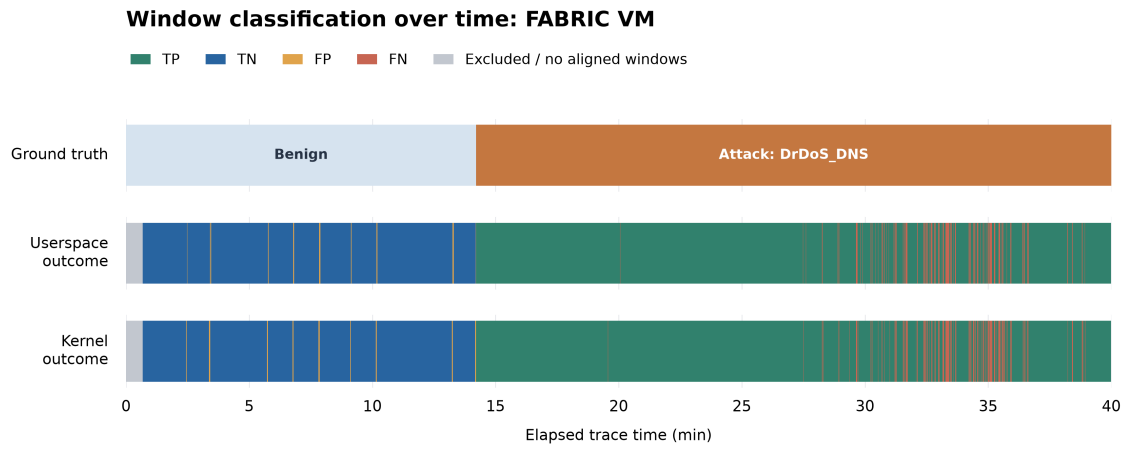


Figure 12: Window classification over time on the FABRIC VM. Grey marks excluded windows. Windows are mapped to trace time by the automatically estimated offsets, which align the onset of the flood but not its later course.

	Userspace	Kernel
Attack-labelled windows	2,881	2,881
Benign-labelled windows	1,512	1,512
True positives (TP)	2,657	2,647
False positives (FP)	33	37
True negatives (TN)	1,479	1,475
False negatives (FN)	224	234
Recall (%)	92.2	91.9
Precision (%)	98.8	98.6
False-positive rate (%)	2.2	2.4
F1 score	0.954	0.951
Accuracy (%)	94.1	93.8

Table 12: Window-level detection quality on the FABRIC VM with blocking disabled. Alignment is approximate (Section 5.1.3).

5.2 Blocking Enabled

	Userspace	Kernel
Blocked Traffic (%)	98.1	98.1
Passed Traffic (%)	0.90	0.95
Dropped at the network interface (%)	1.02	0.97
Block events	49	49
Unique sources blocked	7	7
Windows flagged	154	153
Victim host CPU, whole run (% of all cores)	5.68	5.85
Victim host CPU, flood interval (% of all cores)	10.1	10.4

Table 13: Runs with blocking enabled on the Raspberry Pi 400. Percentages are of the received packets. Blocklist entries expire after 60 s. Block events count new blocklist entries; renewals of entries that are still active are not included.

Raspberry Pi 400 On the Raspberry Pi, 98.1% of the packets are dropped by both implementations, and less than 1% passes on to the detector. Drops at the network interface fall to approximately 1%, and victim host CPU during the flood interval is 10.1% for userspace and 10.4% for the kernel implementation (Table 13). Both runs block the same 7 sources in 49 block events. During the flood interval, traffic is received in bursts approximately every 61 s, when blocklist entries expire before being renewed.

	Userspace	Kernel
Blocked Traffic (%)	99.07	99.08
Passed Traffic (%)	0.919	0.913
Dropped at the network interface (%)	0.00982	0.00571
Block events	47	46
Unique sources blocked	7	6
Windows flagged	139	155
Victim host CPU, whole run (% of all cores)	0.52	0.57
Victim host CPU, flood interval (% of all cores)	1.52	1.32

Table 14: Runs with blocking enabled on FABRIC VM. Percentages are of the received packets. Blocklist entries expire after 60 s. Block events count new blocklist entries; renewals of entries that are still active are not included.

FABRIC VM With blocking enabled, XDP drops 99.07% of the received traffic for userspace and 99.08% for the kernel run. Approximately 0.92% passes, including traffic subsequently lost at the capture socket or excluded by the parser. Drops at the network interface account for 0.00982% and 0.00571%, respectively (Table 14). The flood intervals are 853.6–1581.6 s and 855.2–1583.2 s for the two implementations. Host utilization during those intervals is low for both implementations, at 1.52% and 1.32%.

5.2.1 Memory and Decision Delay

Decision delay is measured from the nominal end of an observation window until its verdicts are available. It includes window closing, scheduling and verdict computation. The kernel-offloaded implementation schedules window closure after a fixed 10 ms grace period, allowing packets stamped

before the boundary to finish updating the maps. The userspace baseline uses the same grace period when closing quiet windows; otherwise, a window closes when a packet from a later window arrives. Consequently, the reported delays include intentional waiting and should be distinguished from verdict computation time.

	Userspace	Kernel
Cgroup main memory, median (MiB)	7.32	7.20
Independent BPF maps (KiB)	150.3	283.1
Cgroup main memory + BPF maps (MiB)	7.47	7.47
Decision delay, median / p95 (ms)	11.53 / 12.50	12.09 / 13.50

Table 15: Run with Raspberry Pi 400, with blocking enabled. The table shows memory metrics using Cgroup and BPF maps, as well as their combined memory footprint between the two implementation models. Decision delay is divided into a median and p95 (95th percentile) value displayed in ms.

Raspberry Pi 400 On the Raspberry Pi, including the BPF maps, both implementations use 7.47 MiB (Table 15). The kernel implementation's median decision delay is on average 0.55 ms longer. Both delays remain below 14 ms at the 95th percentile, against the 536.9 ms between consecutive verdicts.

	Userspace	Kernel
Cgroup main memory, median (MiB)	7.09	7.01
Independent BPF maps (KiB)	119.0	251.6
Cgroup main memory + BPF maps (MiB)	7.20	7.26
Decision delay, median / p95 (ms)	10.69 / 11.22	10.70 / 11.29

Table 16: Run with FABRIC VM, with blocking enabled. The table shows memory metrics using Cgroup and BPF maps, as well as their combined memory footprint between the two implementation models. Decision delay is divided into median and p95 (95th percentile) values displayed in ms.

FABRIC VM Including the recorded BPF maps, median memory is 7.20 MiB for userspace and 7.26 MiB for the kernel implementation (Table 16). Main memory alone is slightly smaller in the kernel run, but its BPF maps are larger: 251.6 versus 119.0 KiB.

Both implementations' median decision delay are almost identical – 10.70 ms for kernelspace and 10.69 ms for userspace. Both 95th-percentile decision delays are below 11.3 ms against the 536.9 ms between consecutive verdicts.

5.3 Summary

	Kernel vs. userspace		
Measure	Raspberry Pi 400	FABRIC VM	Note
Packets counted in windows	73.6 M vs. 21.1 M	78.55 M vs. 78.46 M	whole run
Victim host CPU	16.5 vs. 38.7%	14.2 vs. 37.1%	flood interval
CPU time per counted packet	6.2 vs. 61.7 μs	2.73 vs. 6.69 μs	flood interval
Recall	89.42 vs. 76.02%	91.9 vs. 92.2%	window verdict
False-positive rate	2.25 vs. 2.78%	2.4 vs. 2.2%	window verdict
Victim host CPU, blocking on	10.4 vs. 10.1%	1.32 vs. 1.52%	flood interval
Cgroup memory + BPF maps	7.47 vs. 7.47 MiB	7.26 vs. 7.20 MiB	blocking on
Decision delay	12.09 vs. 11.53 ms	10.70 vs. 10.69 ms	median, blocking on

Table 17: Key observed differences, one run per configuration, blocking disabled unless stated.

RQ1: Resource use Moving the per-packet counting into XDP lowered the host cost of detection on both testbeds. On the FABRIC VM, where both implementations counted nearly all packets at the same rate, host CPU time per counted packet fell from 6.69 to 2.73 μs. On the Raspberry Pi, the baseline counted only 26.75% of the received packets against 93.36%, and CPU time per counted packet fell roughly tenfold, from 61.7 to 6.2 μs. Memory including BPF maps differed by at most 0.06 MiB. The median decision delay was approximately 0.55 ms longer on the Raspberry Pi and almost identical on FABRIC against a 536.9 ms window. With blocking enabled, both implementations used similar host CPU.

RQ2: Detection quality When both implementations received the same traffic, on the FABRIC VM, their detection quality was nearly identical (F1 0.954 versus 0.951). This indicates that the adaptations required by the eBPF verifier did not noticeably affect detection. On the Raspberry Pi, the kernel implementation reached a higher recall (89.42 versus 76.02%). This is consistent with the baseline's packet loss, although the baseline's late start is a confounding factor. Recall of 0.89–0.92 at a precision of about 0.99 is well above the recall of about 0.40 reported by Koshovyi and Wu [7], although the comparison is loose.

6 Discussion

This chapter interprets the results presented in Chapter 5 in relation to the research questions. Section 6.1 summarizes the key findings, and Section 6.2 evaluates how well the experimental design supports them. Section 6.3 describes the limitations of the approach, and Section 6.4 outlines directions for future work.

6.1 Key Findings

The central result of this work is that moving per-packet counting and heavy-hitter tracking into the XDP program lowers the host resources spent on detection. When both implementations receive the same traffic, it does so without a measurable loss in detection quality. How much this matters depends strongly on the hardware. On the Raspberry Pi 400, it decides whether the detector sees the attack traffic at all. On the FABRIC VM, it shows up as lower CPU use for the same amount of work.

Resource use On the Raspberry Pi, the userspace baseline could not keep up with the flood. It counted 26.75% of the received packets, against 93.36% for the kernel implementation, and most of its losses occurred at the capture socket (Table 7, Figure 8). During the flood, the detector process used about one full core (101.6%), while host utilization was 38.7% of all four cores (Table 9). This suggests that the bottleneck was the single-threaded path that reads one packet per system call, not the total CPU capacity of the device. The kernel implementation counted 4.3 times as many packets per second while using less than half the host CPU. CPU time per counted packet therefore fell from 61.7 to 6.2 µs, roughly a tenfold reduction.

On the FABRIC VM, both implementations counted nearly all packets at the same rate (Table 8, Table 10), which makes it the cleaner comparison of cost. For the same work, host CPU during the flood fell from 37.1% to 14.2%, and CPU time per counted packet fell from 6.69 to 2.73 µs. In the kernel implementation, the detector process almost disappears from the profile (0.43% versus 56.1% of one core). This is because the per-packet work runs in software-interrupt context instead, where it is still included in host CPU (Figure 10). All runs used XDP in generic mode, in which the socket buffer is allocated before the program runs (Section 2.2). The savings therefore come from avoiding the copy to a capture socket and the per-packet processing in user space, not from bypassing the network stack. Native mode could be expected to widen the gap further.

Not every metric improved. With blocking enabled, both implementations dropped 98–99% of the received traffic in XDP, and their host CPU use was nearly identical (Table 13, Table 14). Once the heavy sources are blocklisted, almost no packets reach the counting stage in either implementation. Memory use including BPF maps was the same within 0.1 MiB, as seen in 5.3, on both testbeds (Table 15, Table 16). The kernel implementation holds more in BPF maps but slightly less in the process.

The kernel implementation's median decision delay was slightly longer: 0.55 ms on the Raspberry Pi and 0.013 ms on FABRIC. Both implementations nonetheless stayed below 14 ms at the 95th percentile, about 2.5% of the 537 ms window. The time until blocking takes effect is therefore dominated by the window length rather than by the decision delay.

The advantage of the kernel implementation is thus concentrated in the periods when source blocking is not yet, or not fully, in effect:

- the first windows of an attack,
- the bursts that pass when blocklist entries expire, about every 61 s in our runs, and
- attacks whose traffic source blocking does not stop.

Detection quality On FABRIC, where the two implementations received essentially the same traffic, their detection quality was nearly identical (Table 12):

- recall 92.2% versus 91.9%,
- precision 98.8% versus 98.6%,
- F1 0.954 versus 0.951.

This indicates that the adaptations required by the eBPF verifier did not noticeably affect detection (Section 4.2.2). These adaptations were 16-bit fingerprints, separate hashes per row, top-K admission without the exact-plus-one rule, and occasional lost concurrent updates.

On the Raspberry Pi, the kernel implementation detected considerably better: recall rose from 76.02% to 89.42% and F1 from 0.862 to 0.938, with 305 false negatives against 691 (Table 11). Both implementations use the same detection model. The difference is therefore most plausibly explained by packet loss. With almost three quarters of the packets lost before counting, the baseline's attack windows were less likely to exceed the threshold. On constrained hardware, the efficiency of the counting stage thus directly affects detection quality, not only resource use.

On both testbeds, the remaining false negatives are concentrated in the lower-rate traffic after the flood (Figure 11, Figure 12). A volumetric detector is least suited to this part of the trace. In addition, a window is labeled malicious if it overlaps any attack flow, so some attack-labeled windows there may contain little attack traffic.

Koshovyi and Wu [7] reported a recall of about 0.40 and a precision close to 1.0 for the statistical baseline. In comparison, the kernel implementation on both testbeds and the userspace baseline on FABRIC reached a recall of 0.89–0.92, at a precision of about 0.99. The comparison is loose: the dataset and the definition of ground truth are shared, but the excerpt, window length, two-window evaluation and hardware differ. It nonetheless indicates that the statistical detector can reach a substantially higher recall than previously reported. Our Raspberry Pi results suggest that losses on the path to user space may be one contributing factor behind such differences.

6.2 Evaluation

This section assesses how well the experimental design supports the findings in Section 6.1. The resource results and the detection results are considered separately, since they rest on different measurements.

Comparability of the implementations The two implementations share the detection model, address representation, window length, sketch dimensions and IQR configuration, and they were run on the same excerpt on the same testbeds. Differences in system performance can therefore be attributed mainly to where the counting takes place. They are not, however, identical. The kernel implementation hashes differently, uses 16-bit fingerprints, admits top-K candidates without the exact-plus-one rule, and tolerates lost concurrent updates (Section 4.2.2). The comparison therefore measures the two implementations as a whole rather than isolating the effect of moving the counting into the kernel. In addition, the baseline receives packets through an AF_PACKET socket with one system call per packet. Part of the measured gap is therefore due to the capture mechanism, as discussed in Section 6.4.

Resource measurements Using two testbeds of different capacity made it possible to separate two effects. On the Raspberry Pi, overload and packet loss dominate the comparison. On the FABRIC VM, both implementations counted almost the same packets at the same rate, so the remaining difference in CPU reflects per-packet cost alone.

Measuring host CPU including software-interrupt time was essential. The XDP program's work is not charged to the detector process, so comparing process CPU alone (0.43% versus 56.1% of one core on FABRIC (Table 10)) would greatly overstate the gain. Normalizing by counted packets further makes the Raspberry Pi results comparable, even though the two implementations processed very different amounts of traffic. The packet accounting at each stage (NIC, socket, late drops) accounts for all but a few hundred packets per run (Table 7, Table 8), so the losses can be attributed with confidence.

The flood intervals over which CPU was averaged were determined separately for each run. They agree within a few seconds on FABRIC, but on the Raspberry Pi the userspace interval is about 100 s longer, since its counted rate lay close to the detection limit. Decision delay was recorded in all runs, whereas memory was measured only in the blocking-enabled runs. Tables 15–16 report both metrics for the blocking-enabled runs. Since the resource differences are large and consistent across both testbeds, and do not depend on the alignment with the dataset labels, we consider them the most robust results of this work.

Detection measurements The detection metrics are less certain, for two reasons.

1. Alignment. Live windows were mapped to the dataset timeline with a constant offset, adjusted manually at the onset of the flood. This does not guarantee alignment for the whole replay, and the FABRIC alignments did not meet the evaluator's alignment criteria, so those results are provisional (Table 12).
2. Coarse ground truth. A window is labeled malicious if it overlaps any attack flow. This can label windows containing little attack traffic as malicious, and it makes recall and accuracy sensitive to the window length.

The large difference in recall on the Raspberry Pi is consistent with the packet loss and is unlikely to be explained by alignment alone. The small differences on FABRIC, however, do not establish either an advantage or equivalence.

Late start of the Raspberry Pi userspace run. In the userspace run with blocking disabled on the Raspberry Pi, detection started approximately 9.6 minutes into the replayed excerpt and missed most of the initial benign period (Figure 11). As a result, only 432 benign windows were scored against 1,511 for the kernel implementation (Table 11), so the baseline's false-positive rate and accuracy rest on a much smaller sample. The detector also had only about four minutes of benign traffic to build its histories before the flood, which may have affected its verdicts during the attack, although this cannot be separated from the effect of the packet loss. The whole-run CPU figures in Table 9 are not directly comparable for the same reason, while the flood-interval figures and the packet accounting are essentially unaffected.

Repetitions and generality Each configuration was run only once, so run-to-run variance is unknown and no confidence intervals can be given. The kernel implementation's random numbers are not seeded, so its runs cannot be reproduced exactly. The FABRIC VM also uses a virtual function of a shared network card, which may add variation from other users of the infrastructure.

The evaluation also covers a narrow set of conditions:

- one 40-minute excerpt containing a single attack type (DrDoS_DNS),
- replay from a single sender, which did not keep up with the original packet rate during the flood,
- XDP in generic mode only.

The traffic was replayed with tcpreplay with the destination addresses rewritten, and the dataset is several years old. Replayed traces also cannot show how a real attack reacts to being blocked. The results therefore show the relative behaviour of the two implementations under these conditions.

Further work is needed before they can be generalized to other attack types, higher loads, native XDP or live traffic.

6.3 Limitations

Partial In-Kernel Detection As stated in Section 1.1.3, only the per-packet counting is performed in the kernel. Since verdicts are computed in userspace after a window closes, the traffic of an attack's first window still reaches the host, as do the bursts that pass each time a blocklist entry expires (Section 5.2). The evaluation therefore measures the benefit of offloading the counting, not of a complete in-kernel detector.

Simple User-Space Baseline The baseline receives packets through an `AF_PACKET` socket with one system call per packet, without a memory-mapped ring or `AF_XDP`. A more optimized userspace implementation would narrow the gap to the kernel-offloaded one, so the measured improvements should be read as relative to a straightforward userspace detector rather than to the fastest possible one.

Per-Source Detection and Blocking The detector identifies attackers by source address and only evaluates the $K = 10$ heaviest sources per window. An attack spread over more sources than that is blocked gradually, at most K addresses per window, and an attack where no single address stands out is not attributed to any source at all. Blocking by source address also has side effects. A legitimate NAT gateway that aggregates many users can be flagged as a heavy hitter, and since source addresses can be spoofed, an attacker could cause legitimate addresses to be blocked.

Parameters and Warm-Up Several parameters, including the base threshold of 200 packets and the IQR and standard deviation multipliers, are taken from Wickman and Rygaard [6], who chose them for a different dataset and a 1 s window. They have not been calibrated for the traffic or the 537 ms windows used here. In addition, every history needs four samples before it can judge traffic, and windows evaluated during this warm-up are always treated as benign. Judging new IPs against the shared history does not avoid this, since the window history and the shared history both start empty. An attack that is already in progress when the detector starts is therefore not detected during the first few windows, and since warm-up samples are added to the histories, its traffic can raise the thresholds used to judge later windows.

Replayed Traffic and Ground Truth All traffic was replayed from a public dataset using `tcpreplay`, from a single sender and with the destination addresses rewritten. Replayed traces do not capture how a real attack reacts to being blocked, and the dataset is several years old. The ground truth is also coarse: a window is labeled malicious if it overlaps any malicious flow, which can label windows with only a small amount of attack traffic as malicious and makes accuracy and recall sensitive to the window length.

Late Detector Start In one run, the user-space detector on the Raspberry Pi started approximately 9.6 minutes into the replay, which shortened its warm-up and its benign evaluation period. Since each configuration was run only once, this run could not be replaced.

6.4 Future Work

Expanded Dataset Evaluation The current evaluation relies entirely on the CIC-DDoS2019 dataset. As noted in Section 6.3, this dataset is several years old and consists of static, replayed traffic that cannot react to mitigation. Future work should evaluate the detector against more recent and diverse datasets to capture modern DDoS attack vectors and varying network topologies. Furthermore, testing

the system in a live, closed-loop testbed would allow for measuring how adaptive, real-world attacks respond to the dynamic source blocking enforced by the XDP program, providing a more accurate assessment of the detector's practical efficacy.

Fully In-Kernel Detection As discussed in Section 6.3, blocking currently takes effect one window after detection, since the verdicts are computed in userspace. Moving the decision into the XDP program would remove this delay. One approach is to complement the windows with decaying rate estimates. Instead of accumulating counts and clearing them at a boundary, each source keeps a rate estimate that is multiplied by a decay factor depending on the time since its last update, typically exponential, and then incremented [11]. Old packets thus lose weight gradually, and there is no window boundary for an attack to straddle. The update costs a multiplication and an addition per packet. The decay factor could be taken from a precomputed integer table, as with the decay probabilities of HeavyKeeper, and memory could be kept bounded by storing the estimates in the sketch's buckets or in an LRU map. Decaying estimates do not replace the windows, since the IQR rule needs discrete per-interval observations to take quartiles over. They answer whether a source is sending too fast right now, rather than whether the last interval was unusual. Checked in the XDP program, such an estimate could drop the most extreme sources within the first window of an attack, while the window-based detector keeps adapting the threshold to normal traffic.

Integration into a Multi-Layer NIDS The detector in this work is evaluated as a standalone system, but its intended role is to act as the first layer of a larger NIDS, filtering volumetric floods before they reach the machine learning models of the earlier work [7]. Future work should connect the two, for example by exporting the per-window summaries and flagged addresses from the BPF maps to the userspace models. It should then evaluate whether this shielding improves the models' throughput and their detection of the low-rate attacks that heavy-hitter detection does not cover.

Distributed and Spoofed Attacks Detection by source address assumes that individual attackers stand out. In highly distributed or spoofed attacks, the traffic may be spread over so many addresses that none exceeds the threshold, even though the window as a whole is flagged. Hierarchical heavy-hitter detection [10], which aggregates counts over address prefixes such as /24 subnets, could identify the attacking networks when the sources are concentrated in a limited number of prefixes.

Calibration and Evaluation The detector's parameters have not been calibrated for the traffic used here (Section 6.3). A sensitivity analysis of the threshold parameters, the window length and the number of top-K candidates would make the detector easier to deploy on new networks.

Native XDP and AF_XDP Baseline While the FABRIC testbed supported native XDP, the Raspberry Pi 400 evaluation was limited to generic mode due to driver constraints. Future work should evaluate the kernel-offloaded implementation using native XDP [9] across all hardware testbeds. Additionally, comparing this against a more optimized user-space baseline built on `AF_XDP` would help isolate the performance gains of moving the counting logic into the kernel from the benefits of the capture mechanism itself.

7 Conclusion

Userspace DDoS detectors must receive every packet through the kernel network stack before they can count it, and so they compete for the resources an attack aims to exhaust. This work investigated whether moving per-packet counting and heavy-hitter tracking into an eBPF program at the XDP hook reduces the cost of detection without degrading its quality. A userspace baseline and a kernel-offloaded implementation of the same HeavyKeeper and IQR detector were compared on a Raspberry Pi 400 and a FABRIC VM, using replayed CIC-DDoS2019 traffic and XDP in generic mode. Adapting the sketch to the eBPF verifier, with integer decay tables, fixed-size BPF maps, unrolled loops and tolerated concurrent update losses, proved manageable and did not noticeably affect detection.

The kernel implementation consistently used fewer host resources. On FABRIC, where both implementations counted nearly all packets, CPU time per counted packet fell from 6.69 to 2.73 µs. On the Raspberry Pi, the baseline counted only 26.75% of received packets against 93.36%, and CPU time per counted packet fell roughly tenfold, from 61.7 to 6.2 µs. Memory use was equal, and median decision delay was approximately 0.55 ms longer on the Raspberry Pi and almost identical on FABRIC against the 536.9 ms window. With blocking enabled, both dropped 98–99% of traffic in XDP at similar CPU cost, so the benefit lies mainly in the periods before blocking takes effect.

Detection quality was equivalent when both implementations received the same traffic: on FABRIC, F1 was 0.954 for the baseline and 0.951 for the kernel implementation. On the Raspberry Pi, the kernel implementation reached a recall of 89.42% against 76.02%. This is consistent with the baseline's packet loss, although its late start is a confounding factor. On constrained hardware, efficient counting thus affects not only cost but how much of a volumetric attack the detector sees.

These findings rest on one run per configuration, a single attack type, generic XDP, a simple AF_PACKET baseline and a provisional label alignment on FABRIC. Verdicts are still computed in userspace after window closure, with blocklist updates following the verdicts.

Overall, in-kernel heavy-hitter tracking is feasible and substantially lowers the cost of volumetric DDoS detection, most of all on resource-constrained devices. It is not a complete defense, since it does not address low-rate or application-layer attacks. It should instead be seen as a first layer that shields a userspace NIDS and its machine-learning models from the bulk of a flood. Section 6.4 outlines directions for further work.

References

[1] Cloudflare. 2026. *DDoS Threat Report: 2026*. Cloudflare Threat Intelligence. [Online]. Available (access 22.10.2026): `https://blog.cloudflare.com/ddos-threat-report-2026-h1/`

[2] Paul Ruth, Komal Thareja. 2022 *fabrictestbed-extensions* documentation - node. Available (access 26.09.2026): `https://fabric-fablib.readthedocs.io/en/latest/node.html`

[3] Raspberry Pi LTD. April 2026. Raspberry Pi 400 Product Brief. Available (access 26.09.2026): `https://pip-assets.raspberrypi.com/categories/561-raspberry-pi-400/documents/RP-008346-DS-2-raspberry-pi-400-product-brief.pdf`

[4] T. Yang, H. Zhang, J. Li, J. Gong, S. Uhlig, S. Chen, and X. Li, "HeavyKeeper: An accurate algorithm for finding top-k elephant flows," *IEEE/ACM Trans. Netw.*, vol. 27, no. 5, pp. 1845–1858, Oct. 2019. [Online]. Available: `https://doi.org/10.1109/TNET.2019.2933868`

[5] G. Cormode and S. Muthukrishnan, "An improved data stream summary: the count-min sketch and its applications," *J. Algorithms*, vol. 55, no. 1, pp. 58–75, Apr. 2005. [Online]. Available: `https://doi.org/10.1016/j.jalgor.2003.12.001`

[6] A. Wickman and D. Rygaard, "Scalable Anomaly-Based Network Intrusion Detection Using a Statistical Model and Data Sketches: Employing Lightweight Statistical Techniques for Scalable Detection of DDoS Attacks in Network Traffic" Master's thesis, Dept. Comput. Sci. Eng., Chalmers Univ. of Technology and Univ. of Gothenburg, Gothenburg, Sweden, 2025.

[7] O. Koshovyi and S. Wu, "A High-Performance Golang-Based Network Intrusion Detection System" Master's thesis, Dept. Comput. Sci. Eng., Chalmers Univ. of Technology and Univ. of Gothenburg, Gothenburg, Sweden, 2026.

[8] J. Li, D. Maier, K. Tufte, V. Papadimos, and P. A. Tucker, "No pane, no gain: Efficient evaluation of sliding-window aggregates over data streams," *SIGMOD Record*, vol. 34, no. 1, pp. 39–44, Mar. 2005.

[9] T. Høiland-Jørgensen, J. D. Brouer, D. Borkmann, J. Fastabend, T. Herbert, D. Ahern, and D. Miller, "The eXpress Data Path: Fast programmable packet processing in the operating system kernel," in *Proc. 14th Int. Conf. Emerging Networking EXperiments and Technologies (CoNEXT)*, Heraklion, Greece, Dec. 2018, pp. 54–66. [Online]. Available: `https://doi.org/10.1145/3281411.3281443`

[10] G. Cormode, F. Korn, S. Muthukrishnan, and D. Srivastava, "Finding hierarchical heavy hitters in data streams," in *Proc. 29th Int. Conf. Very Large Data Bases (VLDB)*, Berlin, Germany, Sep. 2003, pp. 464–475.

[11] E. Cohen and M. J. Strauss, "Maintaining time-decaying stream aggregates," *J. Algorithms*, vol. 59, no. 1, pp. 19–36, 2006. [Online]. Available: `https://doi.org/10.1016/j.jalgor.2005.01.006`

[12] I. Baldin, A. Nikolich, J. Griffioen, I. I. S. Monga, K.-C. Wang, T. Lehman, and P. Ruth, "FABRIC: A national-scale programmable experimental network infrastructure," *IEEE Internet Computing*, vol. 23, no. 6, pp. 38–47, 2019.

[13] I. Sharafaldin, A. H. Lashkari, S. Hakak, and A. A. Ghorbani, "Developing realistic distributed denial of service (DDoS) attack dataset and taxonomy," in *Proc. Int. Carnahan Conf. Security Technology (ICCST)*, 2019, pp. 1–8.

[14] C. C. Aggarwal, Ed., *Data Streams: Models and Algorithms*. New York, NY, USA: Springer, 2007.

[15] M. Datar, A. Gionis, P. Indyk, and R. Motwani, "Maintaining stream statistics over sliding windows," *SIAM J. Comput.*, vol. 31, no. 6, pp. 1794–1813, 2002.

[16] eBPF Foundation, "What is eBPF?," ebpf.io. [Online]. Available: `https://ebpf.io/what-is-ebpf/`. Accessed: [2026-10-20].

[17] Cilium Authors, "BPF and XDP reference guide: BPF architecture," Cilium Documentation. [Online]. Available: `https://docs.cilium.io/en/latest/reference-guides/bpf/architecture/`. Accessed: [2026-10-20].

A Source Code

Link to Github repo: `https://github.com/apultyn/eBPF-DDoS-Detection`

B Dataset

Link to datasets: CIC-DDoS2019 link: `https://www.unb.ca/cic/datasets/ddos-2019.html`

C TCPreplay configurations

Commands used with tcpreplay:

```
TARGET_IF="enp0s31f6"
PCAP_DIR="/media/ubuntu/Seagate/DAT300-data/CICDDoS2019/PCAP-Rewritten/PCAP-01-12_0-0249"

find "$PCAP_DIR" -maxdepth 1 -type f | sort -V | while IFS= read -r file; do
    echo "========================================="
    echo "Replaying: $(basename "$file")"
    echo "========================================="
    sudo tcpreplay -i "$TARGET_IF" --no-flow-stats --preload-pcap "$file"
done
```

D Final Presentation Slides

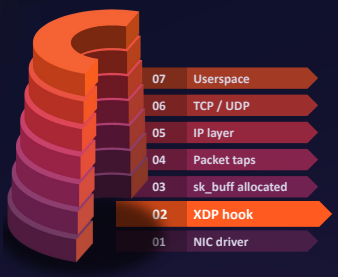
The slides follow on the next page and are read top to bottom, row for row, left to right.

// FINAL PROJECT · CHALMERS 2026

Kernel-Offloaded DDoS Detection with eBPF and XDP

Towards Multi-Layer Intrusion Detection Systems

Ida Åberg · Andrzej Pultyn
Mårten Granath · Tabita Tengblad



// THE PROBLEM

Why move detection into the kernel?

>6×

more attacks above 1 Tbps in Q2 2026 than in Q1

Cloudflare, H1 2026

- 90% of attacks end within 10 min
- Userspace sees packets last
- XDP only blocks, never detects

// RESEARCH QUESTIONS

RQ1 Does it lower the cost of detection?

RQ2 Does detection stay as good?

// RELATED WORK

Building on two earlier theses

2025 · 2026 · 2026

Wickman & Rygaard
Statistical detector

- HeavyKeeper + IQR
- All in userspace

Koshovyi & Wu
ML detector

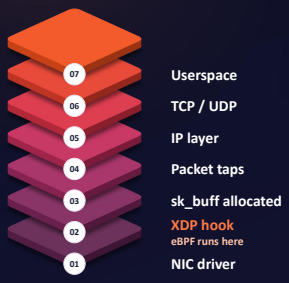
- Isolation Forest
- XDP only blocks

This project
In-kernel counting

- Counting inside XDP
- vs. userspace baseline

// BACKGROUND

eBPF and XDP: act before the network stack



eBPF

Safe programs inside the kernel

XDP

Runs first, on every packet

XDP_DROP · XDP_PASS

// DETECTION BUILDING BLOCKS

Find the heavy hitters, then judge them

HeavyKeeper



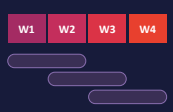
Keeps the top-K source IPs in a tiny, fixed table

IQR threshold



Learns normal traffic, flags what stands out

Windows



537 ms windows, each paired with the one before

// IMPLEMENTATION

Same detector, two places to count



// KERNEL CONSTRAINTS

Getting past the eBPF verifier

No floating point	→ Integer decay table
No dynamic memory	→ Fixed-size BPF maps
Loops must be bounded	→ Unrolled at compile time
Runs on all CPUs at once	→ Accept rare lost updates

// EXPERIMENT SETUP

Two testbeds, one attack excerpt

Raspberry Pi 400

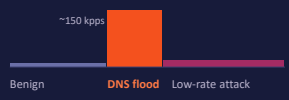
4 × Cortex-A72
1 Gbps Ethernet
Generic XDP

FABRIC VM

2 vCPU AMD EPYC
Shared NIC (mlx5)
Generic XDP

CIC-DDoS2019

40-min excerpt · 78.6 M packets



WE MEASURE

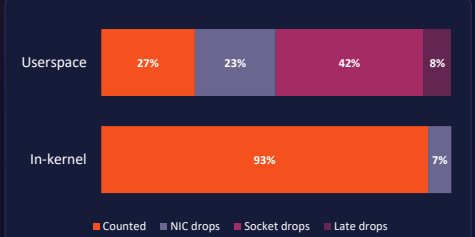
8 runs: 2 testbeds × 2 versions × blocking on/off

Packet loss · CPU · Memory · Delay · Detection

// RESULTS · PACKET LOSS

On the Pi, the userspace detector can't keep up

Where the received packets end up · Raspberry Pi 400, blocking off



3.5×

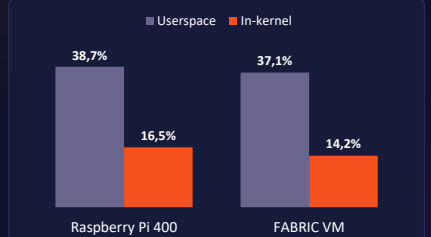
more packets counted in the kernel

FABRIC: both > 99.8%

// RESULTS · CPU

Less CPU for the same work, on both testbeds

Whole-host CPU during the flood, including the kernel time where XDP runs



10×

less CPU per counted packet on the Pi
61.7 → 6.2 µs

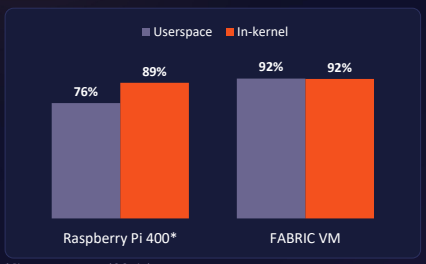
2.5×

less on FABRIC
6.69 → 2.73 µs

// RESULTS · DETECTION QUALITY

Same quality when both see the same traffic

Recall per window (share of attack windows flagged), blocking off



0.954 vs 0.951

F1 score on FABRIC, userspace vs kernel

89–92%

recall, vs ≈ 40% in Koshovyi & Wu

* Pi userspace run started 9.6 min late

// RESULTS · BLOCKING ON

Where the two don't differ

With blocking on, and for memory and decision delay

98–99%

of received traffic dropped in XDP by both

≈ 7.5 MiB

memory for both, BPF maps included

< 14 ms

decision delay, against a 537 ms window

The gain is before blocking takes effect: the first windows of an attack and when blocks expire

// DISCUSSION

Limitations and next steps

Limitations

1. One run, one attack type
2. Generic XDP, simple baseline
3. Verdict still in userspace
4. Only handles large floods

Next steps

1. More runs, more attack types
2. Native XDP and AF_XDP baseline
3. Decide fully in the kernel
4. First layer of the ML-based NIDS

// KEY TAKEAWAYS

- Heavy-hitter tracking fits in the kernel
- 10× less CPU per packet on the Pi, 2.5× on FABRIC
- Same detection quality given the same traffic
- On the Pi: 3.5× more packets, better detection
- A first layer for a multi-layer NIDS

// THANK YOU

Questions?

Ida Åberg · Andrzej Pultyn
Mårten Granath · Tabita Tengblad

github.com/apultyn/eBPF-DDoS-Detection

